Multisets of $O(k)$-invariants under a labelling symmetry group

Working notes

October 15, 2026

1 Setup

Let $L = (v_1, v_2, \ldots, v_n)$ be an ordered list of n vectors, each $v_i \in \mathbb{F}^k$, where $\mathbb{F}$ is usually $\mathbb{R}$ but may be $\mathbb{C}$.

1.1 Invariants

Definition 1 (Dots)**.** For $1 \le i \le j \le n$ let

$$d_{ij} = v_i \cdot v_j = v_i^{\mathsf{T}} v_j.$$

There are $n + \binom{n}{2}$ of these (self-pairs included). The product is bilinear (no complex conjugation) also when $\mathbb{F} = \mathbb{C}$.

Definition 2 (Epsilons)**.** For $1 \le i_1 < i_2 < \cdots < i_k \le n$ let

$$e_{i_1 i_2 \cdots i_k} = \mathrm{Det}\big[v_{i_1}, v_{i_2}, \ldots, v_{i_k}\big].$$

There are $\binom{n}{k}$ of these. The inequalities are strict: any repeated index gives a zero determinant.

For non-canonically-ordered indices we use $d_{ji} = d_{ij}$ and $e_{\ldots j \ldots i \ldots} = -e_{\ldots i \ldots j \ldots}$ (total antisymmetry), so any index list can be brought to canonical (increasing) order, gaining a sign $\mathrm{sgn}(\pi)$ for the sorting permutation π in the case of an e.

1.2 The vector w

Let w be the column vector containing every d and every e in a fixed, predetermined order. Any order is allowed, but it is convenient to keep together the entries that form a single orbit under the permutation part of the group G (below).

1.3 The symmetry group G

G is generated by

- a subgroup of S_n acting on the labels $1, \ldots, n$, typically a product $S(n_1) \times S(n_2) \times \cdots$ permuting labels only within blocks. For example, for $n = 5$, $S(3) \times S(2)$ permutes $\{v_1, v_2, v_3\}$ among themselves and $\{v_4, v_5\}$ among themselves;
- optionally, a parity factor $P = \{1, p\}$, so that $|G \times P| = 2|G|$.

Definition 3 (Parity)**.** p acts on every vector by the same fixed reflection,

$$p : v_i \mapsto R\, v_i, \qquad R = \mathrm{diag}(1, \ldots, 1, -1),$$

that is, it flips the sign of the last component of every vector.

Lemma 1. *For every k, p leaves each d unchanged and negates each e:*

$$p : d_{ij} \mapsto d_{ij}, \qquad p : e_{i_1 \cdots i_k} \mapsto -e_{i_1 \cdots i_k}.$$

Proof. $R^{\mathsf{T}}R = I$, so $(Rv_i)^{\mathsf{T}}(Rv_j) = v_i^{\mathsf{T}}v_j$. Also $\mathrm{Det}[Rv_{i_1}, \ldots, Rv_{i_k}] = \det(R)\,\mathrm{Det}[v_{i_1}, \ldots, v_{i_k}]$ and $\det R = -1$. □

Remark 1. The proof uses only $R^{\mathsf{T}}R = I$ and $\det R = -1$. So any reflection, or more generally any orthogonal matrix with $\det = -1$, acts identically on w, and the choice of which component to flip is immaterial. Because R is real, the d's are invariant under either a bilinear or a Hermitian product. The e's, however, pair naturally with the bilinear d.

Remark 2 (Point inversion is *not* the right parity). The alternative $v \mapsto -v$ (that is, $R = -I_k$) has $\det(-I_k) = (-1)^k$. It negates the e's only for odd k. For even k (for example $k = 2$, where $-I_2$ is a rotation by π) it leaves w completely unchanged. This is why parity is defined as a reflection above.

Remark 3 (Indefinite metrics, for example Minkowski space). The dot products may be taken with any non-degenerate diagonal metric, $d_{ij} = v_i^{\mathsf{T}}\eta\, v_j$ with $\eta = \mathrm{diag}(\pm 1, \ldots, \pm 1)$ (symembed's option `metric`). The determinants e do not involve the metric. Nothing downstream of w depends on it either: every construction of Sections 3 and 4, every output count, and every statement about injectivity on MS is unchanged. Parity is then any isometry of determinant -1, and Lemma 1 holds as before.

What does change is the last step, from MS back to L. Over a definite metric the Gram matrix always determines L up to $O(k)$. Over an indefinite one that is true only when the v_i span a *non-degenerate* subspace. Then equal Gram matrices force equal linear relations among the vectors, so the correspondence between the two configurations is an isometry between their spans, and Witt's theorem extends it to the whole space. (The e's fix the determinant, as before.) The step fails for degenerate spans, which need null vectors. The simplest case is a null vector n and the zero vector, which have the same (zero) Gram matrix.

Example. In $1 + 1$ dimensions, with P in G so that only dot products are needed, take

$$L_1 = \big((3,3),(2,2)\big), \qquad L_2 = \big((3,3),(1,1)\big), \qquad L_3 = \big((3,3),(0,0)\big), \qquad L_0 = \big((0,0),(0,0)\big).$$

Every dot product vanishes in all four. In light-cone coordinates $u = t + x$, $v = t - x$, the non-zero vectors lie on the ray $v = 0$, with $u = (6, 4)$, $(6, 2)$ and $(6, 0)$. A boost multiplies every u by the same factor e^{η}, so the ratio $u_1 : u_2$ is invariant. Parity exchanges u and v, time reversal makes vectors past-pointing, and relabelling only swaps the ratio. So L_1, L_2, L_3 and L_0 lie in four different orbits, even up to G. Yet boosting with $\eta \to -\infty$ shrinks every vector towards zero, so all four orbit closures contain L_0, and every continuous invariant takes the same value on all four. The quantity that does tell L_1 from L_2, the momentum fraction $z = u_1/(u_1 + u_2)$ (0.6 against 0.75), is Lorentz-invariant on exactly-collinear massless pairs but cannot be extended continuously. Near L_0, which both orbits approach, it takes every value. This rhymes with collinear safety in physics: collinear-safe observables are precisely those that do not depend on such splitting fractions.

- *No continuous invariant can do better.* The Lorentz group is not compact, and its orbits need not be closed. Boosts carry n arbitrarily close to 0, so any continuous invariant takes the same value on n as on 0. Every continuous invariant is constant on orbit *closures*, so the most any continuous embedding can do is separate orbit closures.

- *That is also all that is needed.* The purpose of the embedding is that every continuous G-invariant function of L should be a function of the outputs. Continuous invariant functions are themselves constant on orbit closures. So failing to tell apart configurations whose orbit closures meet costs nothing.

- *Physical momenta.* For future-pointing timelike or null vectors, a span containing a timelike vector is non-degenerate, and so is the span of two non-parallel null vectors. So the only degenerate configurations are those in which every vector is null and all are mutually parallel (or zero). Their orbit closures contain 0. Elsewhere, the d's and e's determine L up to $SO(1,3)$, and, because the vectors are future-pointing, up to the proper orthochronous group. Separately, note that the d's and e's cannot detect time reversal: if past-pointing vectors were allowed, invariants of the orthochronous group such as $\operatorname{sign}(p^0)\max(p^2,0)$ would not be functions of them.

For a definite metric every orbit is closed, so none of this arises.[1]

1.4 Group action on w and the multiset

For $g \in G$, the vector $g \cdot w$ is obtained as follows:

1. relabel the indices of each entry according to the permutation part of g;
2. restore canonical index order, using $d_{ji} = d_{ij}$ and the antisymmetry sign for each e;
3. if g contains p, negate every e (Lemma 1).

The resulting entries are re-expressed in terms of the original (canonical) d's and e's, keeping the slot positions of w fixed. Define the multiset

$$\mathrm{MS} = \{\, g \cdot w \mid g \in G \,\},$$

with multiplicities, so that $|\mathrm{MS}| = |G|$ counted with multiplicity.

2 Worked example: $k = 2$, $L = (a, b, p)$

Here $n = 3$. To avoid a clash with the parity element, the vector named p in L is written as a subscript only. There are $3 + 3 = 6$ dots and $\binom{3}{2} = 3$ epsilons. Ordered by orbits of $S(2) \times S(1)$ (bars separate the orbits):

$$w = \big(\, d_{aa}, d_{bb} \;\big|\; d_{pp} \;\big|\; d_{ab} \;\big|\; d_{ap}, d_{bp} \;\big|\; e_{ab} \;\big|\; e_{ap}, e_{bp} \,\big)^{\mathsf{T}}.$$

Let $\sigma = (ab)$. Under σ: $d_{ab} \mapsto d_{ba} = d_{ab}$, $e_{ab} \mapsto e_{ba} = -e_{ab}$, and $e_{ap} \leftrightarrow e_{bp}$ with no sign change.

[1] *Note to self, on [1] and [2].* These papers use the fact that the Lorentz- and parity-invariant polynomials are generated by the dot products (Weyl), together with the density of polynomials, to conclude that every invariant function is a function of the dots. That step needs care for the non-compact Lorentz group. Density of polynomials does not by itself give density of *invariant* polynomials among invariant functions: there is no averaging over a non-compact group. And the conclusion fails if past-pointing momenta are allowed (the $\operatorname{sign}(p^0)$ example above). Moreover, the dots do not determine the momenta up to Lorentz transformations for collinear massless configurations. The conclusions do survive for *continuous* functions of *future-pointing* momenta: continuous invariants are constant on orbit closures, and on that domain the dots separate orbit closures. That justification, rather than the density argument alone, is the one to give.

2.1 $G = S(2) \times S(1)$

$$\mathrm{MS} = \left\{ \underbrace{\begin{pmatrix} d_{aa} \\ d_{bb} \\ d_{pp} \\ d_{ab} \\ d_{ap} \\ d_{bp} \\ e_{ab} \\ e_{ap} \\ e_{bp} \end{pmatrix}}_{1}, \underbrace{\begin{pmatrix} d_{bb} \\ d_{aa} \\ d_{pp} \\ d_{ab} \\ d_{bp} \\ d_{ap} \\ -e_{ab} \\ e_{bp} \\ e_{ap} \end{pmatrix}}_{\sigma} \right\}$$

2.2 $G = S(2) \times S(1) \times P$

$$\mathrm{MS} = \left\{ \underbrace{\begin{pmatrix} d_{aa} \\ d_{bb} \\ d_{pp} \\ d_{ab} \\ d_{ap} \\ d_{bp} \\ e_{ab} \\ e_{ap} \\ e_{bp} \end{pmatrix}}_{1}, \underbrace{\begin{pmatrix} d_{bb} \\ d_{aa} \\ d_{pp} \\ d_{ab} \\ d_{bp} \\ d_{ap} \\ -e_{ab} \\ e_{bp} \\ e_{ap} \end{pmatrix}}_{\sigma}, \underbrace{\begin{pmatrix} d_{aa} \\ d_{bb} \\ d_{pp} \\ d_{ab} \\ d_{ap} \\ d_{bp} \\ -e_{ab} \\ -e_{ap} \\ -e_{bp} \end{pmatrix}}_{p}, \underbrace{\begin{pmatrix} d_{bb} \\ d_{aa} \\ d_{pp} \\ d_{ab} \\ d_{bp} \\ d_{ap} \\ e_{ab} \\ -e_{bp} \\ -e_{ap} \end{pmatrix}}_{\sigma p} \right\}$$

In σp, the swap sign and the parity sign cancel on e_{ab}.

3 Layer 2: embedding the multiset

The multiset MS is not yet a usable output. Next we apply a continuous, injective and algebraic *multiset embedding*

$$\Phi : \left(\mathbb{F}^m\right)^{|G|} / S_{|G|} \longrightarrow \mathbb{F}^D, \qquad m = \dim w,$$

and express each component of $\Phi(\mathrm{MS})$ explicitly as a polynomial in the d's and e's. Several choices of Φ are possible. We start with the Bursarial embedding because its outputs are explicit low-degree polynomials, which makes it the easiest to work with.

3.1 The Bursarial embedding

Index the slots of w from 0 to $m-1$, so $w = (w_0, \ldots, w_{m-1})^{\mathsf{T}}$. For each $g \in G$ define

$$q_g(x) = \sum_{j=0}^{m-1} (g \cdot w)_j \, x^j,$$

and multiply one linear factor in y for each group element:

$$B(x, y) = \prod_{g \in G} \big(y + q_g(x)\big).$$

Elementary symmetric polynomials. For N quantities $z_1, \dots, z_N$, the *c-th elementary symmetric polynomial* is the sum of all products of c of them with distinct indices:

$$E_c(z_1, \dots, z_N) = \sum_{1 \le i_1 < i_2 < \cdots < i_c \le N} z_{i_1} z_{i_2} \cdots z_{i_c}, \qquad E_0 = 1.$$

For example, $E_1 = z_1 + \cdots + z_N$ and $E_N = z_1 z_2 \cdots z_N$. When $N = 3$, $E_2 = z_1 z_2 + z_1 z_3 + z_2 z_3$. They are the coefficients that appear when a product of linear factors is multiplied out:

$$\prod_{i=1}^{N} (y + z_i) = \sum_{c=0}^{N} y^{N-c}\, E_c(z_1, \dots, z_N).$$

(For $N = 2$ this is $(y + z_1)(y + z_2) = y^2 + y(z_1 + z_2) + z_1 z_2$.)

Applying this to B. Take $N = |G|$ and let the z's be the polynomials $q_g(x)$, one for each $g \in G$. Write

$$E_c(x) \equiv E_c\big(\{q_g(x)\}_{g \in G}\big) = \sum_{\substack{\{g_1, \dots, g_c\} \subseteq G \\ (c \text{ distinct elements})}} q_{g_1}(x)\, q_{g_2}(x) \cdots q_{g_c}(x).$$

Then

$$B(x, y) = \sum_{c=0}^{|G|} y^{|G|-c}\, E_c(x).$$

Each q_g has degree (at most) $m-1$ in x, so $E_c(x)$, a sum of products of c of them, is a polynomial in x of degree (at most) $c(m-1)$. Write its coefficients as $E_{c,j}$:

$$E_c(x) = \sum_{j=0}^{c(m-1)} E_{c,j}\, x^j.$$

(The notation $[x^j]\, f$ is sometimes used for the coefficient of x^j in f, so that $E_{c,j} = [x^j]\, E_c$.) The leading term $y^{|G|}$ ($c = 0$) carries no information. The embedding is the list of all the remaining coefficients:

$$\Phi(\mathrm{MS}) = \big(E_{c,j}\big)_{1 \le c \le |G|,\ 0 \le j \le c(m-1)},$$

which has

$$D = \sum_{c=1}^{|G|} \big(c(m-1) + 1\big) = |G| + (m-1)\, \frac{|G|(|G|+1)}{2}$$

components. This is the construction used in Echinocoder's `Cinf_sympy_bursar_embedder_for_array_of_reals_as_multiset.py`, with $n \to |G|$ and $k \to m$.

Properties.

- *Injective on multisets.* $\mathbb{F}[x][y]$ is a unique factorisation domain, and each factor $y + q_g(x)$ is monic in y. So B determines the multiset $\{q_g\}$, and hence MS.
- *Invariant.* B is a product over all of G, so $\Phi(\mathrm{MS})$ is unchanged when L is acted on by any $g \in G$.
- *Polynomial.* $E_{c,j}$ is a homogeneous polynomial of degree c in the entries of w, that is, in the d's and e's:

$$E_{c,j} = \sum_{\{g_1, \dots, g_c\} \subseteq G} \ \sum_{j_1 + \cdots + j_c = j} \ \prod_{r=1}^{c} (g_r \cdot w)_{j_r}.$$

- *Order-dependent coordinates.* The slot order of w decides which entries multiply which powers of x. Changing it changes the coordinates of Φ, but not its injectivity.

What injectivity is actually required. We never want to recover L itself. The purpose of MS is to forget exactly the information that G moves. Using d's and e's erases the orientation of L in $\mathbb{F}^k$, that is, the action of $SO(k)$. Over $\mathbb{R}$ the Gram matrix (d_{ij}) determines L up to $O(k)$. The e's then fix the handedness whenever the v_i span $\mathbb{F}^k$; when they do not, there is no handedness to fix. Each factor of G erases something further. For example, $S(3)$ erases which of a, b, c is which, and P, if present, erases handedness, so that L is then known only up to $O(k)$. In that case the e's carry nothing beyond the d's. Every G-invariant is then even in the e's, and by Cauchy–Binet $e_I\, e_J = \det\big[d_{i_r j_s}\big]_{r,s=1}^{k}$. They are kept in w here only for uniformity. A practical implementation would leave them out whenever $P \subseteq G$, because they greatly enlarge MS and everything that then has to be pruned. (Over $\mathbb{C}$ with a bilinear product the degenerate cases, such as nonzero v with $v \cdot v = 0$, still need checking.) Two notions of injectivity should therefore be kept apart:

(A) *Formal injectivity.* Treat the m entries of w as independent variables, and require Φ to separate $\{g \cdot w\}$ for *every* $w \in \mathbb{F}^m$. This is all the Bursarial embedding knows about.

(B) *Injectivity on the multisets that actually occur.* Require

$$\Phi\big(\mathrm{MS}(L)\big) = \Phi\big(\mathrm{MS}(L')\big) \implies \mathrm{MS}(L) = \mathrm{MS}(L'),$$

equivalently $L' = g \cdot (h\, L)$ for some $g \in G$ and orthogonal h (special orthogonal if $P \not\subseteq G$). Only those $w = w(L)$ lying on the variety cut out by the Gram and Plücker relations need to be separated.

(B) is the target. (A) implies (B), but (B) asks less of Φ, so it should be achievable with fewer outputs.

Redundancy. Φ is deliberately oversized, for two separate reasons. First, it does not know that MS is structured. MS is the G-orbit of a single point w, and each slot of $g \cdot w$ is $\pm$ an entry from the same G-orbit block of w, so many coefficients vanish identically and many others are functionally dependent on the rest. Second, it does not know the Gram and Plücker relations among the d's and e's, which are what separate target (B) from target (A). Exploiting both kinds of redundancy is the next layer. The present goal is only the oversized expressions, which are certainly injective, generated algorithmically and verifiably for general G.

3.2 Example: $k = 2$, $L = (a, b, p)$, $G = S(2) \times S(1)$

With the slot order of Section 2, $m = 9$, $|G| = 2$ and $D = 2 + 8 \cdot 3 = 26$:

$$q_1(x) = d_{aa} + d_{bb}x + d_{pp}x^2 + d_{ab}x^3 + d_{ap}x^4 + d_{bp}x^5 + e_{ab}x^6 + e_{ap}x^7 + e_{bp}x^8,$$
$$q_\sigma(x) = d_{bb} + d_{aa}x + d_{pp}x^2 + d_{ab}x^3 + d_{bp}x^4 + d_{ap}x^5 - e_{ab}x^6 + e_{bp}x^7 + e_{ap}x^8.$$

The $c = 1$ block ($E_1 = q_1 + q_\sigma$, 9 coefficients $E_{1,0}, \ldots, E_{1,8}$) is

$$\big(\, d_{aa}{+}d_{bb},\ d_{aa}{+}d_{bb},\ 2d_{pp},\ 2d_{ab},\ d_{ap}{+}d_{bp},\ d_{ap}{+}d_{bp},\ 0,\ e_{ap}{+}e_{bp},\ e_{ap}{+}e_{bp}\,\big),$$

which already shows a vanishing coefficient and repeated ones. The $c = 2$ block ($E_2 = q_1 q_\sigma$, 17 coefficients $E_{2,0}, \ldots, E_{2,16}$) contains, for example,

$$E_{2,12} = 2\big(d_{ap}e_{ap} + d_{bp}e_{bp}\big) - e_{ab}^2.$$

This is where the information about e_{ab} lost from E_1 reappears.

3.3 Involution packing

The Bursarial embedding can be made roughly half the size, while staying injective by construction (Level 0), whenever the multiset is invariant under a fixed linear involution. For MS this is always the case if G contains an element acting on w as a non-trivial involution.

The general mechanism. Let T be a real $m \times m$ matrix with $T^2 = I$, and let M be a multiset of vectors in $\mathbb{R}^m$ that is invariant under T as a multiset, that is, $\{Tv : v \in M\} = M$. Choose integer bases $f_1, \ldots, f_{m_+}$ and $h_1, \ldots, h_{m_-}$ of linear functionals with $f_a \circ T = f_a$ and $h_b \circ T = -h_b$. Here $m_\pm$ are the dimensions of the ± 1 eigenspaces, $m_+ + m_- = m$, and $m_+ - m_- = \operatorname{tr} T$. Pad the shorter list with zero functionals to a common length $r = \max(m_+, m_-) = (m + |\operatorname{tr} T|)/2$, and pack each vector as

$$\phi(v) = \big(f_1(v) + i\, h_1(v),\ \ldots,\ f_r(v) + i\, h_r(v)\big) \in \mathbb{C}^r.$$

Then ϕ is injective, and $\phi(Tv) = \overline{\phi(v)}$, entry by entry. With $q_v(x) = \sum_j \phi(v)_j\, x^j$, the complex Bursarial polynomial

$$B(x, y) = \prod_{v \in M} \big(y + q_v(x)\big)$$

therefore satisfies

$$\overline{B} = \prod_{v \in M} \big(y + q_{Tv}(x)\big) = B,$$

because $v \mapsto Tv$ permutes M. So every coefficient of B is real. $\mathbb{C}[x][y]$ is a unique factorisation domain and each factor is monic in y, so B determines the multiset $\{q_v\}$, hence $\{\phi(v)\}$, hence M. The real coefficients of B are therefore an injective embedding of M with

$$D = |M| + (r - 1)\, \frac{|M|(|M| + 1)}{2}$$

outputs, instead of $|M| + (m - 1)|M|(|M| + 1)/2$.

Application to MS**.** Let $g_0 \in G$ act on w as a non-trivial involution, and take T to be that action. Because $g \mapsto g_0 g$ permutes G,

$$T(g \cdot w) = (g_0 g) \cdot w,$$

so MS is invariant under T and the mechanism applies. The members of MS pair up as $(g \cdot w,\ g_0 g \cdot w)$. Note that g_0 multiplies on the *left*. Pairing $g \cdot w$ with $g g_0 \cdot w$ instead would relate the two by $g g_0 g^{-1}$, which depends on g and is not a single fixed T. When T is a signed permutation, $(Tw)_s = \sigma_s\, w_{\pi(s)}$, integer functionals can be read off slot by slot:

- a fixed slot ($\pi(s) = s$) gives w_s, in the $+1$ part if $\sigma_s = +1$ and in the -1 part otherwise;
- a swapped pair ($\pi(s) = t \neq s$) gives $w_s + \sigma_s w_t$ in the $+1$ part and $w_s - \sigma_s w_t$ in the -1 part.

The best g_0 is the one with smallest $|\operatorname{tr} T|$. The special case $g_0 = p$ (parity) packs every d with an e. The case of g_0 swapping the two members of a zipped pair is symcoder's $z \mapsto i\bar{z}$ compression.

Example. For $S(2)$ acting on (a, b) with $k = 2$, take $g_0 = (ab)$, which has trace 0. Then

$$z_1 = (d_{aa} + d_{bb}) + i\,(d_{aa} - d_{bb}), \qquad z_2 = d_{ab} + i\, e_{ab} = \bar{z}_a z_b,$$

and the 5 outputs are

$$2(d_{aa} + d_{bb}), \quad 2d_{ab}, \quad 2(d_{aa}^2 + d_{bb}^2), \quad 2\big(d_{ab}(d_{aa} + d_{bb}) + e_{ab}(d_{aa} - d_{bb})\big), \quad d_{ab}^2 + e_{ab}^2.$$

The unpacked embedding has 11.

Sizes. The table gives output counts before any reduction.

case	g_0	tr	plain	packed
$S(2)$, $k=2$	(ab)	0	11	5
$S(3)$, $k=2$	(bc)	+1	174	90
$S(2)\times S(1)$, $k=2$	(ab)	+1	26	14
$S(5)$, $k=4$	(de)	+4	138060	79980
$S(3)\times S(2)$, $k=3$	$(bc)(pq)$	+1	1884	948
$S(5)\times P$, $k=4$	$(bc)(de)p$	+2	549720	289440

Limits. Complex conjugation is the only non-trivial symmetry of $\mathbb{C}$ over $\mathbb{R}$, so conjugation alone saves at most a factor of about 2. (Using two commuting involutions in the same way would need an algebra with two independent conjugations. Bicomplex numbers have zero divisors, which breaks unique factorisation, and quaternions do not commute, so the product depends on the order of its factors.) This is *not* a limit on exploiting the group, however. Section 3.4 gets much larger factors by also letting a group element act on the variable x.

On the simple cases of Section 8, full polynomial closure (in L) leaves exactly as many outputs from the packed embedding as from the plain one. The packing does not change what survives that (expensive) reduction. What it gives is the saving for free, with no reduction work, and it does better than the plain embedding followed by the linear (safe) reductions.

3.4 Graded packing

Involution packing let one group element act on the *coefficients* of B, as complex conjugation. Graded packing also lets a group element act on the *variable* x. That kills most coefficients outright, and the saving grows with the order of the element.

A warm-up: even polynomials. Suppose a linear map T permutes the members of a multi-set M, and acts on each vector by flipping the sign of its odd-numbered entries: $(Tv)_j = (-1)^j v_j$. Then

$$q_{Tv}(x) = \sum_j (-1)^j v_j x^j = q_v(-x),$$

and so

$$B(-x,y) = \prod_{v\in M} \big(y + q_v(-x)\big) = \prod_{v\in M} \big(y + q_{Tv}(x)\big) = B(x,y),$$

because $v \mapsto Tv$ only reorders the factors. So B is an even function of x, and every coefficient of an odd power of x is zero for every input. Those coefficients never need to be stored. This is the same factor of 2 as involution packing, obtained with real numbers by putting the $+1$ coordinates of T at even positions and the -1 coordinates at odd positions.

Roots of unity. Now replace -1 by $\zeta = e^{2\pi i/r}$. If a symmetry h of M multiplies the coordinate at position j by ζ^j, then $q_{hv}(x) = q_v(\zeta x)$, so $B(\zeta x, y) = B(x,y)$. Comparing the coefficients of $y^{|M|-c}x^j$ on both sides gives $E_{c,j} = \zeta^j E_{c,j}$, and $\zeta^j \neq 1$ unless r divides j. Hence

$$E_{c,j} = 0 \quad \text{unless } r \mid j,$$

so only about one coefficient in r survives.

Finding such coordinates. For MS, take any $h \in G$, and let r be the order of its action on w. Since $h^r = 1$, every eigenvalue of that action is an r-th root of unity ζ^k. Call $k \in \{0, \dots, r-1\}$ the *weight*. Use eigen-coordinates: linear functionals p with $p(h \cdot v) = \zeta^k p(v)$. They are real when $\zeta^k = \pm 1$, and complex otherwise. Put each coordinate of weight k at a position $j \equiv k \pmod r$, and leave unused positions empty (zero). A complex eigen-coordinate of weight k and its conjugate (weight $-k$) carry the same information, so only one of each pair is kept. It can be used with weight k or, conjugated, with weight $r - k$.

Shifting the residues. The positions need not be congruent to the weights themselves. Put every coordinate of weight k at a position $j \equiv k + \delta \pmod r$, for one fixed *shift* δ. Then $\zeta^k = \zeta^{j-\delta}$ at every occupied position, so $q_{hv}(x) = \zeta^{-\delta} q_v(\zeta x)$, and

$$B(x, y) = \prod_v \big(y + \zeta^{-\delta} q_v(\zeta x)\big) = \zeta^{-\delta|M|} B(\zeta x, \zeta^\delta y).$$

Comparing coefficients of $y^{|M|-c} x^j$ gives $E_{c,j} = \zeta^{\,j-\delta c} E_{c,j}$, so

$$E_{c,j} = 0 \quad \text{unless } j \equiv \delta c \pmod r.$$

Still about one coefficient in r survives, but the shift changes where the coordinates sit. That matters, because the number of coefficients of $E_c(x)$ is $c(W-1)+1$, where W is the length of q (the highest occupied position plus one, taking the lowest to be 0). So the total is about $\sum_c c(W-1)/r$, and W should be as small as possible. If the busiest weight has M coordinates, they need positions $j_0, j_0{+}r, \dots, j_0{+}(M{-}1)r$, so $W \geq (M{-}1)r{+}1$. Equality needs all the busiest weights to sit at the very start of the first block of r positions: in general $W = (M-1)r + t$, where t is the length of the shortest run of consecutive residues (cyclically) containing the shifted residues of all the weights with M coordinates. The shift δ is chosen to achieve this, and the weight-k or weight-$(r-k)$ choice for each conjugate pair is made to keep the busiest weights few and adjacent. For a fixed shift the pairs do not interact, so the code tries every shift and splits each pair to minimise its own last position; every layout is a translate of one found this way.

Example: for $S(3) \times S(2)$ with $k = 4$ ($r = 6$), without a shift the weights 2 and 3 had two coordinates each, with the second copies at positions 8 and 9, so $W = 10$. The code now uses weight 4 rather than 2 for one conjugate pair, so that the busiest weights are 3 and 4, and the shift $\delta = 3$ puts them at positions 0 and 1 (and 6 and 7). Then $W = 8$, and the count falls from 124 to 100.

Making the coefficients real. The surviving coefficients are complex in general, and would cost two reals each. But suppose G also contains an involution s with $shs = h^{-1}$. Then h and s generate a dihedral group, like the rotations (h) and reflections (s) of a regular r-gon. The coordinates can then be chosen so that also

$$p(s \cdot v) = \overline{p(v)}.$$

(Both s and complex conjugation map the weight-k space to the weight-$(-k)$ space. Their composite is a "real structure" on each weight space, and coordinates fixed by it have exactly this property.) For the real weight spaces ($k = 0$, and $k = r/2$ if r is even) this just means pairing an s-even coordinate with an s-odd one as (even) + i (odd), as in involution packing. Then $\overline{B} = B$, as in Section 3.3, and every surviving coefficient is real.

Invariants, and the scalar mode. Coordinates fixed by every element of G, such as $d_{aa}{+}d_{bb}$, are equal for every member of MS. They can be output directly and left out of the packing. Occasionally h then multiplies *every* remaining coordinate by the same factor ζ^ρ. In that case the coordinates are placed consecutively and the grading moves to y: from $q_{hv} = \zeta^\rho q_v$ it follows that $E_c = \zeta^{\rho c} E_c$, so whole blocks E_c vanish unless r divides ρc.

Exact zeros from symmetries of single coordinates. The grading is one instance of a more general argument, which also finds coefficients that vanish for subtler reasons. Expanding the product,

$$E_{c,j} = \sum_J F_J,$$

where J runs over *patterns*: multisets $\{a_1, \ldots, a_c\}$ of occupied positions with $a_1 + \cdots + a_c = j$. Here F_J collects the products $\prod_{g \in S} \phi_{a_g}(g \cdot w)$ over all c-element subsets $S \subseteq G$ and assignments $g \mapsto a_g$ whose positions form the multiset J. So F_J depends only on the multiset of tuples $\{(\phi_a(g \cdot w))_{a \in J}\}_{g \in G}$, and it has degree n_a in the a-th entries, where n_a is the number of times a occurs in J.

Now suppose some $t \in G$ multiplies each coordinate in the pattern by a scalar: $\phi_a(t \cdot v) = \chi_a \, \phi_a(v)$ for every $a \in J$. Because $g \mapsto tg$ permutes G, that multiset of tuples is unchanged when every a-th entry is multiplied by χ_a. So

$$F_J = \Big(\prod_a \chi_a^{n_a}\Big) F_J,$$

and $F_J = 0$ unless $\prod_a \chi_a^{n_a} = 1$. If every pattern of $E_{c,j}$ is killed in this way, then $E_{c,j}$ vanishes for every input.

- Taking $t = h$, which scales every coordinate, gives back the grading.
- A coefficient with no patterns at all is an unreachable power.

The code finds, exactly in $\mathbb{Z}[\zeta_L]$, which group elements scale which coordinates. It then examines every coefficient within a small distance of either end of $E_c(x)$, where the patterns are few because most positions must equal the end position.

Example: for $S(3)$ with $k = 3$, position 0 holds the purely imaginary coordinate $6i\, e_{abc}$. The reflection s multiplies it by -1, so every pattern of $E_{c,0}$, which can only be $\{0, \ldots, 0\}$, picks up the factor $(-1)^c$. Hence $E_{c,0} = 0$ for every odd c. Equivalently, over the 6 group elements the coordinate takes the values $\pm 6i\, e_{abc}$ three times each, and $E_{c,0}$ is a coefficient of $(y^2 + 36e_{abc}^2)^3$. That takes the case from 20 to 17 outputs.

Since only the coordinates at the two ends matter here, the code also chooses which coordinate of the lowest weight goes first and which of the highest goes last: among the shortest layouts, every choice involving coordinates with non-trivial characters is tried, and the one with fewest outputs kept. For $S(5)$ with $k = 4$ that gives 60 zeros (13141 → 13082). (With the shifted layout the ends of the $S(3) \times S(2)$ cases hold coordinates without such characters. Without the shift, $i\, e_{abc}$ sat at position 0 for $k = 3$ and 12 coefficients vanished, but the longer q cost far more.)

Why this is Level 0. The packing (together with any invariants output directly) is an injective linear map from w. The complex Bursarial embedding is injective on multisets of complex vectors, by unique factorisation in $\mathbb{C}[x][y]$. The omitted coefficients are zero for every input, so they carry no information, and when s is used the imaginary parts of the kept coefficients are zero too. (Besides the powers killed by the grading, the code also omits any $E_{c,j}$ whose j cannot be written as a sum of c *occupied* positions. When some positions are empty padding, these are zero for the simplest of reasons: the product of c polynomials q_g contains no other powers of x.) Everything is exact. In practice the code tries one h from each conjugacy class, every suitable s (or none), and with or without taking the invariants out, and keeps whichever choice gives the fewest outputs. The choice $h = e$ recovers involution packing.

Example: $S(2)$**,** (a, b)**,** $k = 2$**.** Here $w = (d_{aa}, d_{bb}, d_{ab}, e_{ab})$. The invariants $d_{aa} + d_{bb}$ and d_{ab} are output directly. The swap $h = (ab)$ multiplies both remaining coordinates, $d_{aa} - d_{bb}$ and e_{ab}, by -1, so this is the scalar mode with $r = 2$. Packing them as $z = (d_{aa} - d_{bb}) + i\, e_{ab}$, the multiset is $\{z, -z\}$ and $B = (y + z)(y - z) = y^2 - z^2$. So $E_1 = 0$, and the only coefficient is $-z^2$. It is complex, because no element of G acts on z as conjugation. The four outputs are

$$d_{aa} + d_{bb}, \qquad d_{ab}, \qquad \operatorname{Re} z^2 = (d_{aa} - d_{bb})^2 - e_{ab}^2, \qquad \operatorname{Im} z^2 = 2\, e_{ab}(d_{aa} - d_{bb}).$$

This is the hand-made zip of Section 8, and it equals the count left by the full polynomial closure. The plain embedding has 11 outputs. (The code builds its coordinates from integer rows, here $z = (d_{aa} - d_{bb}) + 2i\, e_{ab}$, and outputs the coefficient $-z^2$. That carries the same information. The explorer `symembed/explore/simple_cases_graded.py` writes the graded embeddings of the simple cases out in full, with exact coefficients.)

Example: $S(3)$**,** (a, b, c)**,** $k = 2$**.** Take $h = (abc)$ ($r = 3$) and $s = (bc)$. The 9 slots of w give three weight-0 coordinates and three complex coordinates of weight ± 1.

- The weight-0 coordinates are $\sum d_{ii}$ and $\sum_{i<j} d_{ij}$, which are s-even, and one combination of the e's, which is s-odd. Paired, they become 2 complex coordinates, placed at positions 0 and 3.
- The three complex coordinates are split between residues 1 and 2, at positions 1, 4 and 2.

So each q has length $W = 5$, and E_c has degree $4c$. Keeping only powers divisible by 3 leaves $\lfloor 4c/3 \rfloor + 1$ real coefficients for each $c = 1, \ldots, 6$, that is $2 + 3 + 5 + 6 + 7 + 9 = 32$, against 174 for the plain embedding.

Sizes. In the table below, the "plain" and "involution" columns count every coefficient of those constructions. The "graded" column counts the outputs of the graded construction itself. Coefficients that vanish identically (by the grading, as unreachable powers, or by character symmetries) are part of that construction, and are already omitted. The last column also applies the optional exact Level 1 reductions (`reductions="safe"`).

case	(h, s)	plain	involution	graded	factor	+ safe
$S(2)$, $k = 2$	(ab), scalar	11	5	4	2.8	4
$S(2) \times S(1)$, $k = 2$	(ab), scalar	26	14	11	2.4	11
$S(3)$, $k = 2$	(abc), (bc)	174	90	24	7.2	24
$S(4)$, $k = 2$	$(abcd)$, (bd)	4524	2124	613	7.4	612
$S(4)$, $k = 3$	$(abcd)$, (bd)	3924	2124	386	10.2	385
$S(3) \times S(2)$, $k = 3$	$(abc)(pq)$, (bc)	1884	948	165	11.4	164
$S(3) \times S(2)$, $k = 4$	$(abc)(pq)$, (bc)	1494	870	100	14.9	100
$S(5)$, $k = 4$	$(abcde)$, $(be)(cd)$	138060	79980	13082	10.6	13081

Limits.

- *The factor is bounded by the dihedral subgroup used.* The saving is roughly $|\langle h, s \rangle| = 2r$, and r is at most the largest order of an element of G (6 for $S(5)$, 30 for $S(10)$). That is tiny compared with $|G| = n!$. This is a large constant factor for moderate n, not a change in the $|G|^2$ growth of any Bursarial construction.
- *Unbalanced weights cost some of the gain.* When the weights are unevenly spread over the residues, the busiest residue sets the length of q, even after the best shift (for example, D_4 gives 7.4× rather than 8 for $S(4)$, $k = 2$).

- *Exact further reductions.* The packing is built exactly, with coefficients in the cyclotomic integers $\mathbb{Z}[\zeta_L]$ for $L = \mathrm{lcm}(r, 4)$, which contain both ζ_r and i. Each number is stored as a short integer vector (length $\phi(L)$: 2 for $L = 4$, 4 for $L = 12$, 8 for $L = 20$), and the floating-point coordinates used for evaluation are derived from these. With `reductions="safe"` two exact Level 1 searches run.

  - *The $c = 1$ block.* Each $E_{1,j}$ equals $(p_j S)\, w$, where $S = \sum_g M_g$ and p_j is a weight-0 coordinate built from integer rows. Any that is a rational combination of the invariant outputs or of earlier ones is dropped.
  - $2 \le c \le c_{\max}$. The outputs are expanded exactly in $\mathbb{Z}[\zeta_L]$. As an exact check of the construction, every omitted coefficient is verified to vanish and every kept one to be real. Then each output P contributes the rows $P, \zeta P, \ldots, \zeta^{\phi(L)-1} P$ as integer vectors ("restriction of scalars"). Rational linear relations among these rows are exactly the linear relations among the outputs with coefficients in $\mathbb{Q}(\zeta_L)$, so the integer relation search applies unchanged.

  As predicted, little is found beyond the symmetry zeros above: $S(4)$, $k = 2$ goes from 613 to 612 and $S(5)$, $k = 4$ from 13082 to 13081 ($c_{\max} = 3$).

- *Zeros beyond $c_{\max}$.* The work budget stops the search above at a modest $c_{\max}$ (3 or 4 in the cases above), but a few outputs with larger c vanish identically. They are found in two steps: numerics propose, exact arithmetic decides.

  - *Proposal.* Each kept output with $c > c_{\max}$ is evaluated exactly modulo two large primes $p \equiv 1 \pmod{L}$, at random points w, with ζ_L replaced by a primitive L-th root of unity modulo p. That replacement is a ring homomorphism from $\mathbb{Z}[\zeta_L]$ to the integers modulo p, so an output that is identically zero evaluates to zero. A nonzero polynomial of degree c vanishes at a random point with probability at most c/p, so the outputs that evaluate to zero at both points are very likely zero. That is not a proof.
  - *Proof.* The proof is an exact expansion of that one coefficient $E_{c,j}$. Only positions that occur in some choice of c occupied positions (repetition allowed) summing to j can contribute to it. Restricting every q_g to those positions therefore leaves $E_{c,j}$ unchanged, and near either end of the range of j there are only one or two such positions, involving few slots. So the expansion is cheap even when c is large. (It is done modulo enough primes that their product exceeds twice a bound on the size of the integer coefficients, which makes it exact.)

  This drops $\mathrm{Re}\, E_{5,32}$ for $S(3) \times S(2) \times P$, $k = 4$ (100 to 99), $\mathrm{Re}\, E_{5,57}$ for $S(3) \times S(2)$, $k = 3$ (165 to 164), and $\mathrm{Re}\, E_{5,75}$ for $S(3) \times S(2)$, $k = 2$ (233 to 232). The other cases in the table have no such zeros. (Before the shift was introduced it also dropped two outputs for $S(5)$; with the present choice of end coordinates those are among the character-symmetry zeros.) Exact evaluation also shows that the many outputs at large c which merely *look* zero in floating point (for $S(4)$ and $S(5)$) are not zero. They are tiny compared with the largest output with the same c, and a relative threshold mistakes them for zero.

  Why such zeros occur is illustrated by $E_{5,42}$ for $S(3) \times S(2) \times P$, $k = 4$, in the layout without shift (positions congruent to weights). It shows a cancellation that the character argument above cannot see. Only positions 8 and 9 contribute, with $z_9 = 2\sum_i (d_{ip} - d_{iq})$ and z_8 the weight-2 combination $2[(d_{ap} + d_{aq}) + \omega(d_{bp} + d_{bq}) + \omega^2(d_{cp} + d_{cq})]$, $\omega = e^{-2\pi i/3}$. For $g = (\sigma, \tau) \in S(3) \times S(2)$, $z_9(g) = \pm u$ depends only on τ, and $z_8(g) = \beta_\sigma$ only on σ, taking the values $\omega^k \beta$ (σ even) and $\omega^k \gamma$ (σ odd). Then $E_{5,42}$ is the coefficient of $s^2 t^3$ in

$$\prod_g \bigl(1 + s z_9(g) + t z_8(g)\bigr) = \prod_\sigma \bigl((1 + t\beta_\sigma)^2 - s^2 u^2\bigr),$$

namely $-u^2$ times the t^3 coefficient of $\prod_\sigma(1+t\beta_\sigma)^2 \cdot \sum_\sigma(1+t\beta_\sigma)^{-2} = \big(1+2t^3(\beta^3+\gamma^3)+\cdots\big)\big(6-12t^3(\beta^3+\gamma^3)+\cdots\big)$, which is $12(\beta^3+\gamma^3)-12(\beta^3+\gamma^3)=0$.

- *No further gain from substitutions.* Scaling x, scaling y, reversing $x \mapsto 1/x$, and conjugation appear to be the only substitutions compatible with $\prod(y+q)$. So beyond about $2r$ (perhaps $4r$), this route offers nothing more.

3.5 Nested coset packing

Every Bursarial construction so far treats MS as one multiset of $|G|$ vectors, and its cost grows like $|G|^2$. Part of that growth can be removed by splitting MS along a subgroup.

The construction. Let H be a subgroup of G. Every $g \in G$ can be written uniquely as hg', with $h \in H$ and g' a fixed representative of the right coset Hg. So MS is the disjoint union, over the $M = |G|/|H|$ right cosets, of the H-orbits

$$H \cdot (g' \cdot w) = \{\, hg' \cdot w : h \in H \,\}.$$

Each H-orbit is itself a multiset, of $|H|$ vectors, invariant under H. It can therefore be embedded by graded packing for H (Section 3.4), giving one vector of some length D_H per coset. The multiset of these M vectors is then embedded by a plain Bursarial embedding. The outputs do not depend on which coset representatives are chosen, and they are G-invariant, since replacing w by $g \cdot w$ only permutes the cosets.

Why this is Level 0. The outer Bursarial embedding is injective on multisets of vectors, so it determines the multiset of inner vectors. The inner embedding is injective on H-orbits, so that determines the multiset of H-orbits. Their union is MS.

When it helps. The outer stage costs about $D_H M^2/2$. For a dihedral H of order $2r$, graded packing gives $D_H \approx m|H|/2$, so the total is about $m|G|^2/(4|H|)$: roughly half of the flat graded count, but only when there are many cosets. With few cosets the generic outer stage, and the loss of G's structure inside each coset, make it worse. The code (embedder `bursar_nested`) searches dihedral subgroups H and uses nesting only if it beats the flat graded packing; otherwise it returns the latter.

case	flat graded	nested	best H
$S(3)$, $k=2$	24	(44)	flat used
$S(3)\times S(2)$, $k=3$	165	(209)	flat used
$S(3)\times S(2)\times P$, $k=4$, e's kept	364	299	order 12, 2 cosets
$S(5)$, $k=4$	13082	5455	$S(2)\times S(3)$, 10 cosets

Limits.

- The outer stage is a plain Bursarial embedding. Grading it as well would need some element normalising H to act *linearly* on the inner outputs. In general it acts by relabelling packed coordinates, which changes the inner coefficients non-linearly.
- Nesting again, along a chain of subgroups, gains only about a further factor of 2 per level, and only when each level has many cosets.
- The degrees are no lower: inner outputs have degree up to $|H|$ in w, and the outer stage multiplies up to M of them, giving degree up to $|G|$, as before. For $S(3)\times S(2)\times P$ with $k=4$, unit-scale inputs already give Jacobian entries near 10^{46}.
- The exact Level 1 searches are not applied to nested embeddings.

3.6 Cascaded graded embedding

Graded and nested packing still treat MS as a multiset of $|G|$ (or $|H|$) vectors, each of which has to be computed by applying a group element to w. For a dihedral group there is a better way. Let $D = \langle h, s\rangle$ be a dihedral subgroup of G ($shs = h^{-1}$; or $D = \langle h\rangle$ cyclic, without s), and let $z(w)$ be the packed coordinates of a graded packing for D (Section 3.4): $z_a(h \cdot w) = \zeta_r^{k_a} z_a(w)$ and $z_a(s \cdot w) = \overline{z_a(w)}$. Then the whole D-orbit of the packed vector is known from $z(w)$ alone, by multiplying by roots of unity and conjugating, and it can be embedded in stages.

A cyclic stage. Let g generate a cyclic group Z_R (at first $g = h$ and $R = r$), and let the current coordinates Y_i be eigen-coordinates: $Y_i(g \cdot w) = \zeta_R^{\kappa_i} Y_i(w)$, with $Y_i(s \cdot w) = \overline{Y_i(w)}$. Pick a factor p of R and the subgroup Z_p generated by $g^{R/p}$, which multiplies Y_i by $\zeta_p^{\kappa_i}$.

- Coordinates with $p \mid \kappa_i$ are fixed by Z_p. They are passed on unchanged, with weight κ_i/p for the quotient $Z_{R/p}$.
- The others are placed in one polynomial $Q(x) = \sum_i Y_i x^{j_i}$, at positions $j_i \equiv \kappa_i + \delta \pmod R$. (A coordinate may be replaced by its conjugate, which has weight $-\kappa_i$; the shift δ and these choices make Q as short as possible, exactly as in Section 3.4.) The Z_p-orbit of these coordinates is the multiset of the p polynomials $Q_t(x) = \sum_i \zeta_p^{\kappa_i t} Y_i x^{j_i}$, and its Bursarial polynomial $\prod_{t=0}^{p-1}(y + Q_t(x))$ determines it, by unique factorisation. Only the coefficients $E_{c,j}$ with $j \equiv \delta c \pmod p$ survive.

The congruence modulo R (not just modulo p) is what makes the next stage possible: $\zeta_R^{\kappa_i} = \zeta_R^{j_i - \delta}$, so $Q_t(g \cdot w)(x) = \zeta_R^{-\delta} Q_t(w)(\zeta_R x)$, and therefore $E_{c,j}(g \cdot w) = \zeta_R^{j-\delta c} E_{c,j}(w)$. So the outputs of the stage are again eigen-coordinates, with weights $(j - \delta c)/p$ modulo R/p, and s still conjugates them (it conjugates every Q_t). The next stage treats them in the same way.

The conjugation stage. After stages along a factorisation $r = p_1 p_2 \cdots$, every coordinate is fixed by $\langle h\rangle$, and the outputs so far determine the $\langle h\rangle$-orbit of the packed vector. The pair $\{Y, \overline{Y}\}$ then determines the D-orbit; it is embedded by $\operatorname{Re} Y$ and the coefficients of the real polynomial $Q\overline{Q}$, $Q(x) = \sum_i Y_i x^i$ (the Bursarial polynomial $(y+Q)(y+\overline{Q})$). Coordinates known to be purely imaginary need no real part. Without s, $\operatorname{Re} Y$ and $\operatorname{Im} Y$ are output.

Why this is Level 0, and nesting. Each stage is injective on the orbits it embeds, so the outputs determine the D-orbit of w. When $D = G$ that is MS. Otherwise the D-orbits of $g \cdot w$, one per right coset Dg, are embedded this way and their multiset by a plain Bursarial embedding, as in Section 3.5. The code (`embedder="bursar_cascade"`) searches D (one h per conjugacy class, a few s for each) and the chain, and keeps the smallest.

Sizes. Each stage multiplies the number of coordinates by about $(p+1)/2$, against about $|D|/2$ for a flat Bursarial embedding of the orbit. Outputs, with the best flat graded or nested count for comparison:

case	graded/nested	cascade	D, chain
$S(3)$, $k = 2$	24	21	D_3, 3
$S(3)$, $k = 3$	17	12	D_3, 3
$S(3) \times S(2)$, $k = 2$	232	109	D_6, $3 \cdot 2$
$S(3) \times S(2)$, $k = 3$	164	95	D_6, $3 \cdot 2$
$S(3) \times S(2)$, $k = 4$	100	63	D_6, $2 \cdot 3$
$S(3) \times S(2) \times P$, $k = 4$ (e's omitted)	99	52	D_6, $2 \cdot 3$
$S(4)$, $k = 3$	297	219	D_4, $2 \cdot 2$, 3 cosets
$S(4)$, $k = 4$	249	153	D_4, $2 \cdot 2$, 3 cosets
$S(5)$, $k = 4$	5455	3420	D_6, $2 \cdot 3$, 10 cosets

For $S(3) \times S(2)$ with $k = 4$ the hand-crafted separating set has 54 variables [1, 2] (26 with parity).

Limits.

- The stages exploit only a dihedral (or cyclic) subgroup: only inside $\langle h \rangle$ do the quotients act on a stage's outputs by substitutions $x \mapsto \zeta x$, $y \mapsto \zeta^{\delta} y$, and s by conjugation. The rest of G is handled by nesting, with its $|G/D|^2$ growth.
- The degrees are no lower: still up to $|D|$ in w per orbit, and $|G|$ with nesting.
- No exact Level 1 search is applied, and coefficients that vanish for subtler reasons (Section 3.4) are not removed.

4 Layer 3: removing redundancy

There is no obviously best method for removing the redundancy in Φ. This section records what we are optimising and which reductions are safe.

4.1 Design objectives

1. *Injectivity on every input.* Target (B) must hold for every L, including degenerate ones: coincident or parallel vectors, and v_i that fail to span $\mathbb{F}^k$. Separation for generic L alone is not acceptable. We expect sets that separate everywhere to be strictly larger than sets that separate only generic L. The extra outputs are needed only to resolve confusions on degenerate inputs. (For indefinite metrics, where some orbits are not closed, read "every input" as "every orbit closure". No continuous map can do more, and nothing more is needed; see Remark 3.)

2. *Few outputs.* The number of outputs is to be minimised subject to (1). It can never be smaller than the dimension of the quotient, which for $n \geq k$ is $nk - \frac{1}{2}k(k-1)$. By Dym and Gortler, about twice the input dimension suffices if dense combinations are allowed, but see (3).

3. *Low complexity of each output, as a function of the raw inputs.* Separating invariants that have high degree and depend on delicate cancellations can be so "spiky" that, plotted against the components of the v_i, they look almost like white noise. Anything that later *looks at* the embedding then cannot generalise from it. So the reduction in (2) is balanced against keeping each output simple. Specifically:

    - Complexity is measured by *degree in the components of the* v_i, not by the size of the expression in d's and e's. Each d has degree 2 and each e has degree k. A finer measure is the multidegree, with one degree for each vector v_i. Every Gram, Plücker and mixed d–e relation is homogeneous, so rewriting through them changes the expression but never the function, and never its degree. The growth in expression size from rewriting $e\,e\,e\,e$ as d's is therefore a problem of how outputs are displayed, not of how smooth they are.
    - Degree does control smoothness. By a Bernstein-type inequality, a polynomial of degree r restricted to the unit sphere has angular gradient at most r times its maximum modulus there.
    - Outputs should be *homogeneous* in L, or better still multi-homogeneous, so that each one scales cleanly with the overall size of L. Splitting an output into its homogeneous parts never harms injectivity, because the parts determine the whole, although it does increase the count. The Bursarial $E_{c,j}$ are homogeneous in L only when $k = 2$.

- A homogeneous output f of degree r can optionally be replaced by $\operatorname{sign}(f)\,|f|^{1/r}$. This is a bijection of $\mathbb{R}$, so injectivity is preserved, and every output then scales linearly with L. The cost is that the output is no longer differentiable where $f = 0$.
- Dense combinations that mix many monomials or degrees, such as generic linear projections, are disfavoured.

4. *Evaluation cost* is a secondary concern and is expected to be modest.

4.2 Reductions that are always safe

Removing an output that is given by an *explicit, verifiable formula* (here, a linear one) in terms of the outputs kept can never harm injectivity, and the formula itself is the proof. The following reductions therefore need no judgement:

- if $P \subseteq G$, omit the e's from w from the outset;
- drop coefficients that vanish identically;
- drop exact duplicates and scalar multiples;
- drop any output in the linear span of those kept. This can be detected formally (in the d's and e's) or as functions of L, where the Gram, Plücker and mixed relations hold automatically.

If the reduced set still separates, then every discarded output is still *determined* by the outputs kept. Equal kept outputs imply equal MS, so a discarded output is a well-defined function of the kept ones. What changes beyond the reductions above is that this function is generally not linear, and not even polynomial. For example, x is determined by x^3 on $\mathbb{R}$, but only through a cube root. Discarded invariants typically satisfy polynomial equations over the kept ones, and injectivity is what selects the correct root. The safety of such a reduction therefore cannot be read off from an identity among the outputs. It needs a certificate valid on every input: either an exact algebraic computation (of the *separating variety*), or an explicit decoding algorithm, with case splits for degenerate inputs, that is proved to recover the orbit of L as in target (B). Tests at random points, such as Jacobian rank or counting preimages, can only propose candidates or reject them.

4.3 Levels of certification

"Certified" means that there is evidence that an output set separates orbits on *every* input (target (B)). That evidence comes in levels of decreasing strength.

Level 0: by construction. Injectivity follows from a general theorem, and nothing needs to be checked for the particular (n, k, G). Example: the unpruned Bursarial embedding (by unique factorisation).

Level 1: explicit identity. A Level 0 set from which outputs have been removed, each removed output being an explicit formula in the kept ones ("0", "= output 7", "$= \frac{1}{2}\,\text{output}_3 - \text{output}_9$"). The code verifies each formula exactly, in rational arithmetic, and injectivity is inherited from Level 0. These are the reductions of Section 4.2.

Level 2: proof of separation. The removed outputs are not formulas of the kept ones, but there is a proof that the kept set separates every input. The proof can take one of three forms:

(a) a theorem proved for a whole family of cases and recorded in these notes, so that the code need only check that the requested (n, k, G) satisfies its hypotheses;

(b) an exact computer-algebra computation for this particular (n, k, G), such as showing that the separating variety is exactly the union of the graphs of the group action. The result is stored as an artefact that can be checked again;

(c) a decoder that reconstructs the orbit of L from the outputs, with a written proof, including case splits for degenerate inputs, that it succeeds on every input. The code implements the decoder, and tests exercise it.

Forms (a) and (c) rest on proofs checked by people, and (b) rests on the correctness of the computer-algebra system. None is formally verified. "Certified" therefore means that a proof exists that we stand behind, not that the proof has been checked by machine.

Level 3: evidence only. Tests at random points pass (full Jacobian rank, correct preimage counts, no confusable pairs found by search), but there is no proof. Generic injectivity is then plausible, but nothing reliable is known about degenerate inputs. Level 3 sets are *uncertified*.

4.4 Composing reductions

Certified reductions (Levels 0–2) compose. Each step leaves a set of outputs that is still injective on MS, so whatever a step discards remains determined, in principle, by what it keeps. The order in which reductions are applied therefore cannot affect correctness. It can only affect what later reductions are able to find: an early, suboptimal reduction may make a better one harder to discover, but it never breaks injectivity. Uncertified (Level 3) candidates carry no such guarantee, and do not compose safely.

Because every step is recorded with its certificate, different orders can be tried from scratch and compared, at least for small cases.

Some reductions prune outputs of a fixed embedding (for example, the safe reductions). Others replace the layer 2 embedding itself by another that is injective by construction (for example, involution packing, below). Later reductions then act on the new outputs, and the chain stays certified.

4.5 A pitfall: pairwise zips do not determine a table

A tempting way to exploit structure is to embed each row of the multiset (viewed as a table whose columns are its members) separately, and then "zip" rows together in pairs, for example by embedding the multiset of complex numbers $u + iv$ formed from two rows. *This is not injective.* Two different tables can agree on every pairwise zip. In the smallest example below, the columns are the members:

$$T: \begin{array}{cccc} A & A & B & B \\ X & Y & X & Y \\ u & v & v & u \end{array} \qquad T': \begin{array}{cccc} A & A & B & B \\ X & Y & X & Y \\ v & u & u & v \end{array}$$

Every pair of rows has the same multiset of column pairs in T as in T', but (A, X, u) is a column of T and not of T'. A less symmetric example is

$$T: \begin{array}{ccccccc} A & A & B & B & B & C & C \\ X & Y & X & Y & Z & Y & Z \\ v & w & w & v & u & u & w \end{array} \qquad T': \begin{array}{ccccccc} A & A & B & B & B & C & C \\ X & Y & X & Y & Z & Y & Z \\ w & v & v & u & w & w & u \end{array}$$

This counterexample stopped the development of the "lumped multiset zip" idea in the Set Encodings Redux notes.

MS is not a general table: it is a G-orbit. In an orbit, the values of a slot orbit appear equally often, which neither example above respects. But on degenerate inputs, coinciding values merge

those counts into unequal ones. So orbit structure cannot be assumed to rescue pairwise (or r-wise) zips. Any scheme built on partial zips needs a proof, valid on every input, that uses the orbit structure.

5 Collider events: using the beams to fix a frame

At a collider the two incoming protons p and q are known in advance: both are forward timelike with $p^2 = q^2 = m_p^2$, and $p \cdot q$ is fixed by $\sqrt{s}$, which changes only between run periods. Their order is irrelevant, so there is an $S(2)$ exchanging them. The $S(3) \times S(2)$ examples of [1, 2] are exactly this situation, with (pq) the protons and three exchangeable visible particles. Treating p and q as two more vectors (Section 3.4 and after) works, but the embedding then spends outputs separating information about p and q that never changes. This section uses the beams to fix a frame instead.

5.1 The beam frame and what is left of the Lorentz group

Work in the centre-of-mass frame of $p + q$, with p along $+z$: the *beam frame*. A Lorentz transformation that maps the set $\{p, q\}$ to itself fixes $p + q$, so it is a rotation of this frame (or a reflection, if those are allowed), and it either fixes the z axis pointwise or reverses it. So what is left is:

- rotations about the z axis, which fix p and q;
- the *swap*: a rotation by π about any transverse axis, for example $(p_x, p_y, p_z) \mapsto (p_x, -p_y, -p_z)$, which exchanges p and q;
- if the embedding is to be blind to handedness, reflections: in a plane containing the z axis ($p_y \mapsto -p_y$, fixing p and q), or in the transverse plane ($p_z \mapsto -p_z$, exchanging them).

Together with permutations of identical visible particles (a product of symmetric groups, one per species, as before), that is the whole symmetry group.

5.2 The vector w

Rotations about z are removed by using invariants. For visible particles $1, \ldots, n$ with beam-frame momenta $(E_i, p_{x,i}, p_{y,i}, p_{z,i})$, write $t_i = (p_{x,i}, p_{y,i})$ for the transverse momentum, and

$$w = \big(m_1, \ldots, m_n,\ p_{T,1}, \ldots, p_{T,n},\ (i \cdot j)_T \,\big|\, p_{z,1}, \ldots, p_{z,n},\ (i \wedge j)_T\big),$$

with $p_{T,i} = |t_i|$, the transverse dot products $(i \cdot j)_T = t_i \cdot t_j$ and the transverse wedge products $(i \wedge j)_T = p_{x,i}p_{y,j} - p_{y,i}p_{x,j}$, for $i < j$. The part after the bar is called the *bracket*.

Lemma 2. *For every input with $E_i > 0$ for all i, w determines the visible momenta up to a rotation about the z axis.*

Proof. $E_i = (m_i^2 + p_{T,i}^2 + p_{z,i}^2)^{1/2}$. If every t_i is zero there is nothing more to find. Otherwise pick a with $t_a \neq 0$ and rotate so that $t_a = (p_{T,a}, 0)$. Then $t_j = \big((a \cdot j)_T, (a \wedge j)_T\big)/p_{T,a}$ for every j. □

No genericity is assumed: coincident, collinear or beam-aligned particles are covered.

How the symmetries act on w. The swap leaves the dots unchanged and negates every p_z and every wedge (reflecting the transverse plane reverses its orientation): it negates exactly the bracket. Exchanging identical particles permutes the slots, with a sign on a wedge whose order is reversed. A reflection containing the beam axis negates the wedges only; a reflection in the transverse plane negates the p_z's only. So the symmetry group acts on w by signed permutations, and everything in Sections 3.4 to 3.6 applies unchanged, with this w in place of the dots and determinants.

Parameters, not outputs. $\sqrt{s}$ does not appear in w: it is the same for every event of a run, and is stored separately. The same goes for masses that are fixed in advance: a species whose particles all have one known mass (for example massless particles, or muons) has no mass slots, and that mass is a parameter of the embedding. Species whose masses vary from event to event (jets, say) have one mass slot per particle, and larger embeddings as a result.

Missing transverse momentum. Optionally, one transverse-only object can be added. It contributes its p_T and its dots (and wedges) with every other transverse momentum, and no p_z or mass. It is fixed by the permutations; the swap reflects it like everything else.

Blind to handedness: no wedges needed. With reflections in the group, the wedges can be omitted (Level 0), just as the determinants could when $P \in G$ (Section 3.4 and before). The transverse dots, together with $p_{T,i}^2 = (i \cdot i)_T$, form the Gram matrix of the t_i, which determines them up to $O(2)$: up to a rotation about the beam axis and possibly a reflection containing it. That reflection is now a symmetry, so nothing is lost. What remains of the bracket is the p_z's, still up to a common sign because of the swap.

Masses or masses squared? w stores m_i by default; m_i^2 is an option (`mass_variable="m2"`). Both determine the event for physical particles ($m_i \geq 0$), but they behave differently elsewhere.

- $m_i^2 = E_i^2 - |\vec{p}_i|^2$ is a polynomial in the four-momentum and allows $m_i^2 < 0$, which reconstructed objects can have. With m_i, such inputs have to be clipped to $m_i = 0$, so they are not separated from each other, and $\sqrt{\cdot}$ has an infinite derivative at $m_i = 0$, where massless particles sit.
- If masses are computed from $(E, \vec{p})$, rounding in $E^2 - |\vec{p}|^2$ is large for light, energetic particles: in float32, a 1 TeV massless particle can acquire $|m^2|$ of order $10^{-7}\,\mathrm{TeV}^2$, that is $|m|$ of a few hundred MeV. This affects m and m^2 alike, but only m also inherits the infinite slope.
- m_i is the natural physics variable, scales like the momenta, and is what detector-level inputs (p_T, η, ϕ, m) provide directly, with no root to take.

Since the choice is a single option, nothing has to be rewritten to change it later. But changing it changes the outputs, so anything trained on one setting has to be retrained for the other.

5.3 Handling the swap

There are two ways to deal with the exchange of the beams, and the code offers both (option `swap`).

(B) *As one more group element* (default). The multiset is twice as large, but the swap is central and acts by -1 on the bracket, so the graded, nested and cascaded packings use it at little cost. For three identical particles the group is $S(3) \times Z_2 \cong D_6$, as for $S(3) \times S(2)$ before, with the swap in the role of (pq).

(A) *By storing the bracket up to sign.* Echinocoder's embedder of $\mathbb{R}^k/\{\pm 1\}$ stores b up to sign as the coefficients of $-q(x)^2$, $q(x) = \sum_a b_a x^a$ (the Bursarial polynomial of $\{b, -b\}$, with $2k-1$ numbers). On its own, with a plain Bursarial embedding of the multiset over the permutations, this is a baseline: the output of $q(x)^2$ does not change linearly when particles are permuted, so no graded packing can be used. In a cascade whose first stage is the subgroup generated by the swap, that first stage *is* this embedder, but with the bracket's coordinates placed according to their weights under the permutations, so that later stages are still graded.

Sizes. Outputs for some choices of visible particles (graded with the safe reductions; parity rows with the wedges omitted):

visible particles	slots	graded	nested	cascade	cascade, swap first	(A) plain baseline
2 identical	8	14	14	13	15	29
2 identical + missing p_T	13	25	25	23	25	50
3 identical	15	84	84	47	47	405
3 identical, fixed mass	12	56	56	40	40	342
3 identical, blind to handedness	12	81	81	38	38	279
4 identical	24	2953	1654	914	974	9624
2 + 3 (two species)	35	901	716	362	539	3756

For three visible particles the quotient dimension is $4 \cdot 3 - 1 = 11$. Treating the beams as vectors gave 63 (cascade) and 52 with parity; the beam frame gives 47 and 38, against 54 and 26 for the hand-crafted sets [1, 2], which in addition assume timelike or null momenta.

5.4 Inputs

The evaluator takes beam-frame inputs of shape $(\ldots, n_{\text{objects}}, 4)$, columns (m, p_x, p_y, p_z): the form in which all its computations are natural. (The mass column is ignored for fixed-mass species and for missing p_T.) Front ends convert from other conventions:

- lab-frame four-momenta (E, p_x, p_y, p_z), when the lab frame is the beam frame;
- detector coordinates (p_T, η, ϕ, m);
- four-momenta in any frame together with the beams p and q (useful with a crossing angle). The beam frame is built from them: time axis $e_t = (p+q)/\sqrt{s}$, e_z along the part of p orthogonal to e_t, e_x by Gram–Schmidt from a lab axis (an arbitrary choice, on which the outputs do not depend), and e_y completing a right-handed frame. This front end also returns $\sqrt{s}$.

6 Code interface (draft for discussion)

This section specifies what code in Echinocoder should do, before any code is written. It is intended to be iterated on.

6.1 Two phases: build once, evaluate many times

Build (once, may be slow). The user supplies a specification and receives an embedding function together with its metadata:

```
embed_fn, meta = build_invariant_embedding(k, group_spec, options)
```

- `k`: the dimension of each vector.

- `group_spec`: defines n, the labels, and G. G is a direct product of symmetric groups, one for each *species*, for example $((a, b), (p))$ for $S(2) \times S(1)$, together with a flag for whether P is included. This matches symatom's existing `VectorType`/`Context`, which already builds w (as `repS`) and the signed-permutation action, and should be reused. Parity is not yet a group element in symatom. This class of groups is sufficient for now. Arbitrary permutation subgroups are out of scope, but the design should not rule them out.
- `options`: control the construction (see below).

Symbolic work happens only at build time: constructing w and G, forming the embedding, applying reductions, and recording provenance.

Evaluate (many times, must be cheap). The target framework is PyTorch, as used by the IWPC package (`https://github.com/jjhwilkinson/iwpc`, which depends on `torch` and `lightning`). `embed_fn` is therefore a `torch.nn.Module`:

```
Y = embed_fn(X)      # X.shape == (..., n, k)  ->  Y.shape == (..., D)
```

- Any leading batch dimensions ("`...`") are allowed, and may differ from call to call. n and k are fixed at build time, and inputs with any other trailing shape are rejected.
- Along the n axis, `X[..., i, :]` is the vector with the i-th label of `group_spec`. That ordering convention is part of the specification.
- Internally the pipeline is: `X` $\to$ the d's and e's (w, shape $(\ldots, m)$) $\to$ the polynomials chosen at build time $\to$ optional root transform $\to$ `Y`. Only tensor operations are used, so autograd works throughout. No group elements are enumerated at evaluation time. Any constants are registered as buffers, so that `.to(device)` and `state_dict()` behave as usual.
- `Y` must separate orbits for *every* input, and must be differentiable almost everywhere, so that it can be used for backprop. Differentiability may fail at corners: for example, the root transform is not differentiable where $f = 0$. Its gradient there must be defined explicitly (finite, rather than `inf` or `nan`).
- *IWPC compatibility, without depending on IWPC.* Echinocoder must not have a hard dependency on IWPC. It should, however, fit naturally into IWPC, using the same packages (`torch` and `numpy`) and the same conventions as IWPC's `Encoding` base class:
  - an `nn.Module` whose `input_shape` and `output_shape` are registered as integer buffers, and which acts on tensors of shape `(..., *input_shape)`;
  - `input_shape` $= (n, k)$ and `output_shape` $= D$;
  - a flattened variant with `input_shape` $= nk$ (vector-major: component c of v_i at position $ik + c$), because IWPC's `&` concatenation needs vector inputs;
  - NumPy-style docstrings, as in IWPC.

  Wrapping the module as an IWPC `Encoding` should then be a matter of a few lines. Echinocoder may ship such an adaptor, provided IWPC is imported only when the adaptor is used, so that nothing else in Echinocoder needs IWPC installed.
- NumPy inputs may be accepted for convenience, for example in tests and documentation, by conversion to tensors.

6.2 Options (initial list)

- `embedder`: which multiset embedding to use (layer 2), chosen from a registry so that others can be added. Currently `"bursar"` (Section 3.1), `"bursar_involution"` (involution packing, Section 3.3) and `"bursar_graded"` (graded packing, Section 3.4), all Level 0. Graded packing subsumes involution packing, which is kept as the simpler illustration. There is also `"bursar_nested"` (Section 3.5), which nests graded packing over the cosets of a subgroup when that is smaller, and otherwise returns the graded embedding.

- `reductions`: which reductions to apply, as an ordered list or a level. Early versions may support only `none`. Next would come the safe reductions of Section 4 (zero, duplicate, scalar multiple, linear span, which are Level 1; and dropping the e's when $P \subseteq G$, which is Level 0 for the smaller w, because the d's alone determine L up to $O(k)$). Level 2 reductions are applied only if the corresponding proof, artefact or decoder exists in the code. The main builder never returns a Level 3 set. See Section 4.3.

- `homogenise`: split outputs into homogeneous (or multi-homogeneous) parts in L.

- `root_normalise`: apply $\mathrm{sign}(f)|f|^{1/r}$ to each output that an exact counting argument proves homogeneous of degree r in L. This is now implemented. With P in G every output qualifies ($S(3) \times S(2) \times P$, $k = 4$: the largest output falls from about 5×10^{22} to about 14, and float32 tracks float64 roughly ten times more closely). One caveat: a root amplifies rounding noise near $f = 0$. Outputs that vanish identically as functions of L are pure rounding noise, and would become order-one garbage. The build therefore leaves unnormalised any output that is numerically zero at random inputs. (For the case above that used to be $E_{5,42}$. It is *formally* zero, and is now dropped by the search for zeros beyond $c_{\max}$ in Section 3.4. With the shifted layout of that section the case has 99 outputs, none of them left unnormalised.)

- `field`: $\mathbb{R}$ initially. Whether to support $\mathbb{C}$ is still open.

- `dtype`: the precision of evaluation, `float32` by default or `float64`. It affects only the evaluator. The build-time symbolic work is exact, with integer or rational coefficients. In keeping with the usual `torch` conventions, the module can also be converted afterwards with `.to(dtype)`.

An embedding that works in `float64` but not in `float32` is already too fragile to be useful. Keeping outputs of low degree and low noise (objective 3 of Section 4) is partly what makes `float32` sufficient, so the designs should aim to work in `float32`. As a diagnostic, the build could compare `float32` and `float64` evaluations on random inputs and record the discrepancy for each output in `meta`.

6.3 Metadata

`meta` is for documentation and auditing only. It is never used in evaluation. It should be serialisable (for example as JSON), and should contain or allow generation of:

- the spec and options it was built from, plus the Echinocoder version or commit, so that the embedding can be rebuilt and checked;

- D, together with the slot order of w;

- for each output: its symbolic formula (see below), its degree and multidegree in L, whether it is homogeneous, any transform applied, and its provenance (for example, "$E_{2,12}$ of the Bursarial embedding");

- a log of the reductions: each discarded output, the reason, and the evidence. Examples are "identically zero", "equal to output i", "$= \sum_i \lambda_i$ (output i)", or a named certificate;
- the *certification level* (Section 4.3) and what it rests on: the theorem, the identities, the stored artefact, or the decoder together with its proof;
- a LaTeX table generator for the outputs.

Symbolic form of the outputs. Evaluation works from the components of L, but the symbolic description never expands below the d's and e's. These are its leaves. Nobody reading `meta` needs to see $EF - p_x q_x - p_y q_y - p_z q_z$ in place of a dot product. Degrees in L are still reported, counting 2 for each d and k for each e.

The description may also use *nested definitions*, naming intermediate quantities once and referring to them by name. For example,

$$\mathrm{enc}[23] = f\big(h(w)\big) + g\big(h(w)\big), \qquad \text{with } h(w) = \dots$$

Natural intermediates are $q_g(x)$, the elementary symmetric polynomials E_c and their coefficients $E_{c,j}$, and orbit sums. This applies to both the machine-readable form (a set of named definitions, such as a DAG of expressions) and the LaTeX output (a table of outputs plus a table of definitions). Its purpose is to keep each displayed formula short. Whether to use nesting, and how deeply, is a presentation choice. It does not affect evaluation.

6.4 Placement and reuse

The code lives in a new Echinocoder subpackage next to symatom and symcoder. It should not inherit from the existing `MultisetEmbedder`, which works on one input at a time and has no group specification.

symatom already builds something close to w (`repS`) and the signed-permutation action of G on it, and it should be reused where it matches. It may do exactly what is needed here, or something subtly different. Before any part of it is relied on, its behaviour must be checked against the definitions of Section 1, in particular:

- which atoms appear: all self-dots, cross-dots and e's, and nothing else;
- the canonical index order, and the sign conventions for e's under relabelling;
- parity. symatom's `Atom.sign` is known to conflate the sign from permuting arguments with the sign from spatial parity. This was harmless while there was no parity operation, but P is now being added.

Tests should compare the w and $g \cdot w$ obtained from symatom with an independent, direct implementation of Section 1 on small cases. symcoder's encoders are numerical, mostly based on sorting, and are not Bursarial. So the main thing to reuse there is perhaps its design experience rather than its code.

6.5 Scale

The initial goal is $k \le 5$ and $n \le 10$. Larger would be welcome. The goal has strong consequences. For a single species with $n = 10$, $k = 5$:

$$m = \tbinom{10}{2} + 10 + \tbinom{10}{5} = 307, \qquad |G| = 10! \approx 3.6 \times 10^6 \ (\times 2 \text{ with } P), \qquad D_{\mathrm{Bursar}} \approx 2 \times 10^{15},$$

whereas the quotient dimension, which is the lower bound on the number of outputs, is only $nk - \frac{1}{2}k(k-1) = 40$. Hence:

- The unpruned Bursarial embedding is feasible only for small $|G|$. There it serves as a reference baseline, against which reductions can be validated.
- For large G, the build cannot enumerate G, and cannot first form the unpruned output and then prune it. It must construct the reduced outputs directly. One way is to work with orbits of monomials in w (orbit sums), described by the pattern of their labels rather than by listing group elements. symatom's orbit enumerators may help here.
- Evaluation should use tensor contractions, for example $\sum_{i,j} d_{ij}^2 = \mathrm{tr}(\mathbf{d}^2)$, rather than expanded lists of terms, whose length grows with orbit size.

Even $k \leq 5$, $n \leq 10$ may prove too big. It is a goal, not a requirement.

6.6 Uncertified (Level 3) candidates

The main builder returns only sets at Levels 0–2. Level 3 candidates are useful for research, for example to see how small output sets can get and to suggest which sets are worth proving. They are available only through a separate, explicitly named research entry point, *not* through a flag or option on the main builder, so that they cannot be picked up by accident. Their `meta` states that they are uncertified (Level 3), and lists the evidence collected.

7 A needs-based interface (draft for discussion)

This section proposes a user-facing layer on top of the builders of Section 6. Nothing in it is implemented yet. The builders stay as they are: they are the reference implementations, tested on their own, and the layer proposed here only describes, plans and routes.

7.1 Why

The library now serves quite different users:

- collider physicists with exchangeable beams (the LHC), who want invariance under rotations about the beam axis and under the exchange of the beams;
- collider physicists with distinct beams (HERA, or p–Pb at the LHC): an axis, but no exchange;
- users without beams, or who prefer the full treatment of Section 1, which allows spacelike, timelike and null vectors alike;
- users who want no geometric symmetry at all, only invariance under relabelling: raw multisets;
- the same, but whose records contain an angle, such as (p_T, η, ϕ, m), which is not continuous as ϕ wraps round; they need something equivalent to (p_x, p_y, η, m).

At present each of them has to know which builder and which method name fits (`bursar_cascade` means nothing to most of them), may have to reformat inputs outside the library (the last case), and has to find the relevant options. The proposal: users describe their problem; the library lists the methods that solve it, ranked, and builds the one they choose (by default the best).

7.2 What a user describes

A *problem* has four parts, which can be chosen independently.

1. *Particles*: the species, how many particles of each, and per-species declarations (for example, a common mass).

2. *Context*: information about the event that is not one of the exchangeable particles (Section 7.3 below).

3. *Symmetry*: what the encoding should be blind to. Relabelling within each species is always included. On top of it, one of: nothing else; rotations about an axis; rotations about an axis and the exchange of the beams; Lorentz transformations; rotations in k dimensions. Any of these can be made blind to handedness as well.

4. *Inputs*: the format of the user's tensors, for example four-momenta (E, p_x, p_y, p_z), detector coordinates (p_T, η, ϕ, m), (p_x, p_y, η, m), raw features with declared column types, or four-momenta in any frame together with the beams.

Names are neutral, with physics aliases ("axial with beam exchange" is also `LHC`). A sketch, in Python:

```
problem = Problem(
    particles=[Species("jet", 3), Species("lepton", 2, common_mass=True)],
    context=[Context.sqrt_s()],              # the lepton mass is context too
    symmetry=symmetry.axial(beam_exchange=True),        # alias: symmetry.LHC
    inputs=inputs.pt_eta_phi_m)

for c in candidates(problem):                # every method that solves it,
    print(c)                                 # best first: size, degree, level
encoder = candidates(problem)[0].build()     # or simply encoder(problem)
print(encoder.explain())
Y = encoder(particles, context)              # (..., D)
```

Other users change one part only: `symmetry.axial(beam_exchange=False)` (distinct beams), `symmetry.lorentz()` (no beams), `symmetry.none()` (raw multisets; with `inputs.pt_eta_phi_m` the library itself works with $(p_T \cos\phi, p_T \sin\phi, \eta, m)$, so no ϕ-wrap discontinuity can arise, and nothing breaks at $p_T = 0$ either).

7.3 Three kinds of information

Every piece of information about an event is of one of three kinds.

1. *Constants*: the same for every event, ever (for example the proton mass in $p^2 = q^2$). They are declared once, recorded in the metadata, and are neither inputs nor outputs.

2. *Context*: information that can change from event to event (or from run to run) but is not one of the exchangeable particles: $\sqrt{s}$, the common mass of a species, the beam energies, the beams themselves. Context is an input. It may also be used, for example to define the frame (the beams do this) or to stand in for per-particle slots (a common mass replaces the mass slots of its species). And it is *passed through* to the output, so that a change (a new $\sqrt{s}$, three muons instead of three electrons) shows in the encoding instead of passing silently.

3. *Particles*: acted on by relabelling and by the geometric symmetries, and embedded.

The pass-through rule. Context can be passed through *unaltered* only if the symmetries leave it alone. $\sqrt{s}$ and a common mass are unaffected by every symmetry, so they pass through as they are. But if the symmetries act on some context, what is output must be an invariant of it. For example, if the beam exchange is a symmetry and the beam energies may differ, then exchanging the beams exchanges E_p and E_q, and the output must be the unordered pair $\{E_p, E_q\}$ (say, $E_p + E_q$ and $E_p E_q$), not (E_p, E_q). In general the output holds the invariants of the context under whatever the symmetries do to it. Each kind of context therefore declares how the symmetries act on it (invariant by default), and the library chooses the pass-through accordingly. Strictly, the encoding then determines the pair (context, particles) up to the symmetries, which is what is wanted.

Distinct beams. For distinct beams the invariants (m_e^2, m_p^2, s) do not fix the frame: the two beam energies (equivalently, p^μ and q^μ in the frame in which the particles are given) are needed as well, and which frame is meant (the beams' centre-of-mass frame, or the laboratory frame) is part of the description. This case is not a priority, but the design should not exclude it.

7.4 What the library does

Planning. The symmetry and the inputs determine a *front end*: how the particle data become a vector w and how the group acts on it.

symmetry	front end
none (relabelling only)	the features themselves (after continuity-safe conversion)
rotations about an axis (with or without exchange)	per-particle scalars and transverse dots and wedges (Section 5)
Lorentz, or rotations in k dimensions	dots and determinants (Section 1)
blind to handedness	as above, dropping wedges or determinants where proved

Every front end ends with w and a signed-permutation action on it, so every *back end* (plain, involution, graded, nested, cascade) applies. One case is different: with relabelling only, the data already *are* multisets of points, one per species, and embedding them directly involves n points rather than an orbit of $n!$ copies of w. (For three points in $\mathbb{R}^4$, a plain Bursarial embedding needs 21 outputs.) So that case is routed to Echinocoder's existing multiset embedders, one per species, concatenated.

A registry of methods. Each method registers what it can do: which front ends and group structures it accepts, a cheap estimate of its output count and degree made without building it, its certification level, and a rough build cost. The planner keeps every method that fits, estimates, and ranks. A new method is added by implementing and registering it; users then see it among their candidates without any change to their code.

Candidates. `candidates(problem)` returns every method that meets the stated needs, best first, each with its predicted output count, maximum degree, certification level and build cost, and a `build()`. Nothing is built until asked. The default ordering is by output count, among methods at Level 1 or below; degree is reported so that users can trade size against it, and they can try several, since some may suit their use better for reasons that are hard to predict. `encoder(problem)` builds the first. Level 3 candidates never appear here (Section 4.3).

The encoder. The result is a `torch.nn.Module` following IWPC's conventions, whose outputs are the passed-through context invariants followed by the embedding. Its metadata record the problem description, the method chosen, the library version, the certification, the output names, and an `explain()` text: what was chosen and why, what is a constant.

Reproducibility. If the best method changes (because a better one is added), the outputs change, and anything trained on them silently breaks. So the metadata record a method identifier, `encoder(problem, method=meta.method_id)` rebuilds exactly that method, and `explain()` recommends pinning for production use.

Errors. A combination that no method supports produces a message saying which part is unsupported and which nearby problems are supported.

7.5 Questions for review

- Scope: should this layer route raw multisets to Echinocoder's existing embedders (proposed above), and so live at the level of Echinocoder rather than inside `symembed`?
- Inputs: IWPC's `Encoding` takes one tensor. Should context arrive as a second argument, as extra columns, or either (with an adapter that flattens)?
- Context declarations: is "invariant unless declared otherwise" the right default? Which context kinds are needed now (for the LHC: $\sqrt{s}$, common masses; perhaps beam energies for asymmetric runs)?
- Should a common mass always be passed through, or only on request?
- Ranking: output count first, degree as a tie-break or a user choice?

8 Exploration log

This section records what has been observed by running the code on very simple cases. It also lists lines of enquiry those observations suggest. The lines of enquiry are *not* a plan. They are unordered, none is singled out as the next thing to investigate, and other ideas are expected to be added and compared with them.

8.1 Tools and outputs

- `symembed/explore/simple_cases.py` writes each case out in full (`docs/explorations/simple_cases.pdf`, with every expansion in `simple_cases_full.txt`). It shows the slots of w, the action of G, every $E_{c,j}$ expanded in the d's and e's, and the effect of the safe reductions.
- The polynomial-closure tool, `symembed/explore/polynomial_closure.py` (report: `docs/explorations/polynomial_closure.txt`), visits outputs in order of c, then j. It keeps an output unless it is an exact polynomial in outputs already kept, and then retries each kept output against the others ("backward pruning"). Membership is tested either formally in w or as functions of L, where the Gram and Plücker relations hold automatically. Identities are found modulo four primes, reconstructed by the Chinese remainder theorem, and verified exactly. Every kept set is therefore a Level 1 certified separating set: each Bursarial output is an explicit polynomial in it. Each case is run for both the plain and the involution-packed embedding.

8.2 Observations

The table gives the number of outputs in each case (all without parity).

case	Bursarial	linear	poly. in w	poly. in L	quot. dim.
$S(2)$, (a,b), $k=1$	14		6	2	2
$S(3)$, (a,b,c), $k=1$	174		21	5	3
$S(2)\times S(1)$, (a,b,p), $k=1$	26		12	3	3
$S(2)$, (a,b), $k=2$	11	8	5	4	3
$S(3)$, (a,b,c), $k=2$	174	155	32	13	5
$S(2)\times S(1)$, (a,b,p), $k=2$	26	22	15	11	5

1. *Polynomial identities do much more than linear ones*, and allowing the Gram and Plücker relations (working in L rather than formally in w) does much more again.

2. *Low c suffices for generation.* For $S(3)$ every kept output has $c \le 4$ ($k=2$) or $c \le 3$ ($k=1$), although the Bursarial embedding goes up to $c = |G| = 6$. All higher-c outputs are polynomials in lower ones.

3. *For $k=1$* (the classical symmetric-polynomial setting), the count reaches the quotient dimension for $S(2)$ and $S(2)\times S(1)$. For $S(3)$ it stops at 5. The search can only keep actual Bursarial outputs, and it is greedy, so this says nothing about how small a set could be.

4. *Polynomial closure gives generating sets, not minimal separating sets.* For $k=2$ the counts stay well above the quotient dimension (11 against 5, and 13 against 5).

5. *$S(2)$, $k=2$, as a zip.* Here
$$\mathrm{MS} = \{(d_{aa}, d_{bb}, d_{ab}, e_{ab}),\ (d_{bb}, d_{aa}, d_{ab}, -e_{ab})\}.$$
Store the invariant d_{ab} as it is, and zip $u+iv$ with $(u,v) = (d_{aa}, e_{ab})$ in one member and $(d_{bb}, -e_{ab})$ in the other. This gives $d_{aa}+d_{bb}$ (the imaginary part vanishes) and $(d_{aa}d_{bb} + e_{ab}^2) + i\,e_{ab}(d_{bb}-d_{aa})$: 4 reals in all. These coincide with the 4 kept by the polynomial closure, up to a polynomial change of variables. One of the identities needs the Gram relation $e_{ab}^2 = d_{aa}d_{bb} - d_{ab}^2$. The quotient dimension is 3. For fixed $d_{aa}+d_{bb}$ and d_{ab}, the remaining freedom is a closed loop, so any 3-output scheme would have to mix coordinates nonlinearly.

6. *Complex structure for $k=2$.* Identify each v_i with a complex number z_i. Then
$$\bar{z}_i z_j = d_{ij} + i\,e_{ij},$$
so the d's and e's are the real and imaginary parts of products that are invariant under rotation.

7. *Reference covariants reach the quotient dimension generically.*

   - For $S(2)\times S(1)$, (a,b,p), $k=2$: put $\alpha = \bar{p}a = d_{ap} + ie_{ap}$ and $\beta = \bar{p}b = d_{bp} + ie_{bp}$. When $p \neq 0$ the orbit is determined by $|p|^2 = d_{pp}$ and the unordered pair $\{\alpha, \beta\}$, and so by d_{pp}, $\alpha+\beta$ and $\alpha\beta$. That is 5 reals, equal to the quotient dimension, each of degree 2 in w.
     - When $p=0$, $\alpha=\beta=0$ carries nothing. Adding the 4 outputs of $S(2)$, $k=2$ for (a,b) covers that stratum, and d_{pp} shows which case applies. That gives 9 outputs that separate every input, by a short decoder argument (Level 2c), against 11 from the polynomial closure.
   - For $S(3)$, (a,b,c), $k=2$: the sum $s = \sum_i z_i$ gives $\alpha_j = \bar{s}z_j = \sum_i (d_{ij} + ie_{ij})$. The elementary symmetric polynomials of the α_j give $1+2+2=5$ reals, equal to the quotient dimension, but only where $s \neq 0$. The stratum $s=0$ needs something else.
   - In both examples, each degenerate stratum on which the reference vanishes costs extra outputs. This is consistent with the expectation that sets separating every input are larger than sets separating only generic inputs.

Involution packing under reduction. The counts below run from outputs before reduction, to those kept by the polynomial closure in w, to those kept by the polynomial closure in L.

case	plain	involution-packed
$S(2)$, $k=2$	$11 \to 5 \to 4$	$5 \to 5 \to 4$
$S(3)$, $k=2$	$174 \to 32 \to 13$	$90 \to 58 \to 13$
$S(2) \times S(1)$, $k=2$	$26 \to 15 \to 11$	$14 \to 12 \to 11$
$S(2)$, $k=1$	$14 \to 6 \to 2$	$8 \to 6 \to 2$
$S(3)$, $k=1$	$174 \to 21 \to 5$	$111 \to 24 \to 5$
$S(2) \times S(1)$, $k=1$	$26 \to 12 \to 3$	$17 \to 11 \to 3$

After full closure in L the two agree in every case. Formally in w the packed embedding is sometimes worse (58 against 32 for $S(3)$, $k=2$). Packing halves the raw count at no cost, but leaves unchanged what survives the polynomial closure, which is itself only feasible for small cases.

Parity of graded outputs. Each output can be classified by what happens under $e \mapsto -e$, the action of parity: even if unchanged, odd if it changes sign, mixed if neither. Testing numerically, as polynomials in w, with an exact cross-check for low c:

case	outputs	even	odd	mixed
$S(3) \times S(2)$, $k=3$	164	21	1	142
$S(3) \times S(2)$, $k=4$	100	16	0	84

Most outputs are mixed, because the packing combines d's and e's. If outputs of definite parity are wanted, the simplest route is to split each mixed output o afterwards into $\frac{1}{2}(o(L) + o(pL))$ and $\frac{1}{2}(o(L) - o(pL))$, at a cost of one extra output per mixed one. That gives $164 + 142 = 306$ and $100 + 84 = 184$. (The counts in this paragraph before "Splitting is implemented" were made with the earlier layout, without the shift of Section 3.4: then the totals were $191 + 168 = 359$ and $124 + 104 = 228$.) A "parity-graded" packing, using coordinates of pure parity placed so that the parity of each position matches the parity of its coordinate, makes every coefficient pure automatically: $E_{c,j}(p \cdot w) = (-1)^j E_{c,j}(w)$. But it does *worse* here, 395 and 273. The parity condition rules out the best h, whose order 6 is even and conflicts with it, and forces extra padding.

Splitting is implemented as the option `parity_split`. An output is kept as it is when an exact counting argument proves it pure: every way of reaching its power of x uses the same number of e factors modulo 2. Otherwise it is replaced by its even and odd parts.

For graded embeddings the counting argument is refined, using the patterns of Section 3.4: $E_{c,j} = \sum_J F_J$. Parity acts on the coordinate at position a either as a sign (only d's: $+1$; only e's: -1), or, more usefully, as a sign times complex conjugation: $\phi_a(pv) = \pm\overline{\phi_a(v)}$. The latter happens for a coordinate (combination of d's) $+ i$ (combination of e's), which is common, since the real weight spaces pair an s-even coordinate with an s-odd one. If every coordinate of a pattern J is of the first kind, F_J has the product of the signs as its parity, as before. If every coordinate of J is of the second kind, so does F_J: replacing w by $p \cdot w$ conjugates every entry $\phi_a(g \cdot w)$ (up to the signs), and since $\phi_a(sv) = \overline{\phi_a(v)}$ for every packed coordinate, that is the same as relabelling $g \mapsto sg$, which leaves F_J unchanged. So $E_{c,j}$ is pure if all its patterns are determined in one of these two ways with the same parity. For example, when position 0 holds such a coordinate, every $E_{c,0}$ is even. With this refinement `parity_split` gives $184 = 100 + 84$ for $k = 4$, splitting exactly the mixed outputs, and 307 for $k = 3$, against 306: one output, $\operatorname{Re} E_{5,51}$, is odd only through a cancellation that neither argument sees, and contributes an even part that is zero for every input.

Comparison with hand-crafted separating sets. For five Lorentz vectors with $G = S(3) \times S(2)$ ($k = 4$), [1] (Table 3) gives 26 parity-even variables and [2] (conclusions) 28 parity-odd ones. Together they form a separating set of 54 variables. The parity-odd part was crafted by arguments that could not be automated. Those sets are restricted to timelike or null vectors, whereas the constructions here are injective on MS for all inputs, so the comparison flatters the crafted sets somewhat. The fully automatic graded construction (with the safe reductions) gives:

	crafted	here	ratio
with P in G (parity-even only)	26	99	3.8
without P, outputs as they come	54	100	1.9
without P, split into definite parity	54	184	3.4

The quotient dimension is 14. Nonlinear (polynomial) identities and the Gram and Plücker relations have not yet been exploited on graded outputs. On the small cases those were the largest reductions of all.

8.3 Where further savings could come from (October 15–16)

These are observations from a night spent looking for further ways to exploit structure in the packing. As before, the possible next steps they suggest are unordered, and none is singled out.

1. Within "embed the whole orbit", little is left. Graded packing realises the symmetries that a single Bursarial polynomial $\prod(y + q)$ can carry: scaling x, scaling y, and conjugation. (Reversal $x \mapsto 1/x$ would add at most another factor of 2.) That caps the saving at roughly the order of a dihedral subgroup. Nesting over cosets (Section 3.5) adds about a factor of 2 when there are many cosets ($S(5)$, $k = 4$: 13082 → 5455). A second level does not help there: the chain $D_5 \subset A_5 \subset S(5)$ gives 5813 (and $D_5 \subset F_{20} \subset S(5)$ gives 5823), because the middle level has only 2 cosets. Neither changes the $|G|^2$ growth. (Within that cap the layout of the coordinates in q still mattered: choosing the residue shift of Section 3.4 gained 12–25% for $S(3)$, $S(3) \times S(2)$ and $S(4)$, for free.) One size up, $S(6)$ with $k = 4$ ($|G| = 720$, $m = 36$, quotient dimension 18), the plain embedding has 9 085 320 outputs, graded packing 865 682 and nesting 433 770. So for $n \gtrsim 6$ this family cannot be made small, whatever symmetry is exploited.

1a. Cascading along a chain of normal subgroups (now implemented, Section 3.6). When G is itself dihedral, $G = \langle h, s \rangle$ (as $S(3) \times S(2) \cong D_6$ is), the whole orbit of the packed coordinates $z(w)$ follows from $z(w)$ alone: $z(h^t w)$ multiplies each coordinate by ζ^{kt}, and $z(sw) = \overline{z(w)}$. So the orbit can be embedded in stages along a chain such as $\langle h^3 \rangle \subset \langle h \rangle \subset G$. A stage embeds the orbit of the current coordinates Y under a small cyclic group Z_p (the Bursarial polynomial of only p points, graded): coordinates fixed by Z_p are passed on, and the others are placed at positions congruent to their weights plus a shift *modulo the order of the remaining rotation group*. Then the next generator acts on the stage's outputs by the substitution $x \mapsto \zeta x$, $y \mapsto \zeta^\delta y$, that is, diagonally, so its outputs are again eigen-coordinates and the next stage is graded too; and s still acts by conjugation. The last stage embeds the pair $\{Y, \overline{Y}\}$ by $\operatorname{Re} Y$ and the real polynomial $Q\overline{Q}$, $Q(x) = \sum_i Y_i x^i$. Each stage is injective on its orbits by unique factorisation, so the construction is Level 0, and its degree is still at most $|G|$. Exact counts (outputs, including the invariants), against the flat graded counts:

case	flat graded	chain $2 \cdot 3$	chain $3 \cdot 2$
$S(3) \times S(2)$, $k = 4$	100	64	64
$S(3) \times S(2) \times P$, $k = 4$ (e's omitted)	99	52	55
$S(3) \times S(2)$, $k = 3$	165	103	97
$S(3) \times S(2)$, $k = 2$	233	122	110

(The crafted sets have 54 and 26.) These counts were slightly high, because coordinates that are purely imaginary need only half of the last stage; the implementation gives 63, 52, 95 and 109. Each stage costs about $(p+1)/2$ times its input, so along a chain of small primes the size grows like $|G|^{0.6}$ rather than $|G|^2$; but only for chains whose quotients act by such substitutions, which holds inside a cyclic group, plus one conjugation. For larger G the dihedral (or cyclic) part could be cascaded inside nested coset packing: for $S(5)$, $k=4$, inner $S(3)\times S(2)$ at 63 gives $10+62\cdot 55=3420$ against 5455.

2. Graded packing already matches the formal polynomial closure. On the simple cases, the graded counts are close to (sometimes below) what the exact polynomial closure leaves when the d's and e's are treated as independent variables. All the remaining gap comes from the geometry, that is from the Gram and Plücker relations, which hold only as functions of L:

case	graded	closure in w	closure in L
$S(2)$, $k=2$	4	5	4
$S(3)$, $k=2$	24	32	13
$S(2)\times S(1)$, $k=2$	11	15	11
$S(3)$, $k=1$	24	21	5

So further large savings must use the geometry, not just the group. They will have to be nonlinear: in every case of the sizes table up to $S(4)$, the graded outputs that survive the safe reductions, evaluated modulo a large prime at $D+10$ random configurations L (rather than random w), have full rank D. So no output is a linear combination of the others even as a function of L.

3. One geometric lever: dropping slots that are polynomials in other slots. If some slots of w are exact polynomial functions of the others (as functions of L), and the remaining slots are permuted among themselves by G, then the dropped slots can be dropped from w itself. That is a Level 1 reduction, and it shrinks the width of every member of MS. For $k=1$ it is dramatic, since $d_{ij}=e_ie_j$: keeping only the e's gives graded counts of 2, 3 and 3 for $S(2)$, $S(3)$ and $S(2)\times S(1)$, which are the quotient dimensions, and 20 (from 181) for $S(3)\times S(2)$. For $k\geq 2$, however, no slot is a polynomial in the others. The nearest thing is $d_{ij}^2=d_{ii}d_{jj}-e_{ij}^2$ for $k=2$, which fixes d_{ij} only up to sign. So this lever is specific to $k=1$, and to the omission of the e's when parity is in G, which uses the same idea.

4. Parity: an even part plus an odd supplement. Without parity in G, an alternative to splitting every mixed output is possible. Take any embedding that separates orbits of $G\times P$ (99 outputs for $S(3)\times S(2)$, $k=4$). Add the odd parts $\frac{1}{2}(o(L)-o(pL))$ of the graded outputs o (84 non-zero ones). If two configurations agree on the first and on the odd parts, they are either in the same G-orbit or mirror images. In the latter case the odd parts vanish, so the full graded outputs agree, and injectivity of the graded embedding puts them in the same G-orbit after all. That gives 183 against 184, a small saving. What is really needed is only a set of odd invariants whose common zero set (on $G\times P$-orbits) is exactly the set of configurations that are G-equivalent to their mirror image. Odd invariants vanish on hypersurfaces, so several are needed. The hand-crafted solution uses 28 [2], against our 84, which suggests large slack. Certifying a small odd supplement for every input is a Level 2 problem.

5. Beating $|G|^2$ needs something other than a generic multiset embedding. A Bursarial embedding of N points of width W costs about $WN^2/2$, because it cannot know that the points form a single orbit. Two routes avoid this, and both need Level 2 arguments for degenerate inputs.

- *Invariant-theoretic separating sets.* These are orbit sums of chosen monomials, whose number is governed by separating-set theory rather than by N^2. Finding a small one is easy numerically; certifying it for every input is hard.

- *Permutations first, with polynomial frames.* Embed each species' multiset of raw vectors, for example by moment tensors $T_d = \sum_i v_i^{\otimes d}$, $d \leq n_{\text{species}}$, which are rotation-covariant. Then make polynomial covariant frames, such as the Krylov vectors $T_2^a T_1$, $a = 0, \ldots, k-1$, and record every vector's components in that frame. The counts are polynomial in n, but the frame degenerates on special configurations (for example when $T_1 = 0$, or when T_1 lies in an eigenspace of T_2), and each such stratum needs its own treatment.

- *Theorems from the point-cloud literature (Level 2a).* Three iterations of the $(d-1)$-dimensional Weisfeiler–Leman test separate all non-isometric point clouds in Euclidean $\mathbb{R}^d$ [3], and simplex-wise distance distributions are complete isometry invariants computable in polynomial time for fixed dimension [4, 5]. Both are Euclidean only. Making either polynomial, with an injective multiset embedding at every step, makes the dimensions explode: a rough estimate is $\sim 10^8$ outputs for $n = 5$, $k = 4$. So they win only for large n.

8.4 Possible lines of enquiry (unordered, not exhaustive)

- The order of quotienting. The orbit MS has $|G| = n!$ members only because rotations are removed first (via the d's and e's) and the permutations afterwards. Removing the permutations first, by embedding the multiset of the n raw vectors, costs only about $kn^2/2$ outputs. But the result is then covariant, not invariant, under rotations, and the difficulty moves into rotation invariants of it. For $k = 2$ these are phases: the c-th elementary symmetric function of the z_i has weight c. That route would need Level 2 arguments for degenerate inputs. The point-cloud literature has completeness theorems (for example simplex-wise distance distributions, and $(d-1)$-WL tests for d-dimensional clouds) that might supply them.

- Extending the safe reductions from linear to polynomial identities (Level 1). On the cases above this alone takes, for example, 26 down to 11.

- Reference covariants, for $k = 2$ via the complex structure, with the degenerate strata handled by further covariants or by recursion to smaller cases. Open questions: whether there is an analogue for $k \geq 3$, which has no complex structure, and how the number of strata grows.

- Understanding why low c suffices for generation, and whether there is a useful bound on the c needed in terms of the structure of G.

- Further simple cases for comparison: parity, $k = 3$, and two non-trivial species.

- Complex packing of w: pair the real slots of w into $\lceil m/2 \rceil$ complex numbers, embed the multiset of packed vectors with the complex Bursarial embedding, and write out real and imaginary parts. The raw count changes very little, because the Bursarial size is linear in m. For even m it saves $|G|(|G|-1)/2$ reals (as already noted in the legacy embedder's docstring), which is a fraction of about $1/m$. For odd m, padding with zero costs $|G|$ reals. Any real benefit would have to show up in what survives reduction, and that may depend on the pairing (the order of the slots). A factor of 2 would need an element of G acting on the packed vector as complex conjugation, and without parity no permutation other than the identity does so.

- Involution packing, which generalises the $z \mapsto i\bar{z}$ trick and does not need parity. Choose any involution $g_0 \in G$. It splits w into a $+1$ part and a -1 part. A slot pair (s, t) swapped

by g_0 contributes $w_s + w_t$ and $w_s - w_t$, with the signs adjusted as required, and a fixed slot contributes to the part given by its sign. Pack the $+1$ coordinates as real parts and the -1 coordinates as imaginary parts, padding with zeros if the counts differ. Then g_0 acts on every packed vector as complex conjugation. The multiset of packed vectors is therefore closed under conjugation, so $\prod_g (y + q_g(x))$ has real coefficients, and only those need storing.

  - This is Level 0: the complex Bursarial embedding is injective by unique factorisation, and the packing is a bijection.
  - The number of packed slots is $m'' = (m + |\operatorname{tr} g_0|)/2$, so the involution with smallest $|\operatorname{tr}|$ is best.
  - Raw counts fall to roughly half: $11 \to 5$ for $S(2)$, $k = 2$; $174 \to 90$ for $S(3)$, $k = 2$; $26 \to 14$ for $S(2) \times S(1)$, $k = 2$; $138060 \to 79980$ for $S(5)$, $k = 4$; $1884 \to 948$ for $S(3) \times S(2)$, $k = 3$.
  - Now implemented (Section 3.3, embedder `bursar_involution`). What survives full polynomial closure is unchanged on the simple cases (see above). The factor of 2 has since been exceeded by graded packing (Section 3.4).

- Graded packing (Section 3.4, embedder `bursar_graded`): a dihedral subgroup acting on x by roots of unity and on the coefficients by conjugation. It gives 7–15× on the cases tried ($S(5)$, $k = 4$: $138060 \to 13082$). Open questions:

  - what survives polynomial closure;
  - exact Level 1 reductions over $\mathbb{Q}(\zeta_r)$;
  - whether other structures of G can be exploited in the same way.
  - Going beyond a factor of 2 with two commuting involutions would need an algebra with two independent conjugations. Bicomplex numbers have zero divisors, which breaks unique factorisation, and quaternions do not commute, so the product depends on the order of its factors.

References

[1] B. Gripaios, W. Haddadin and C. G. Lester, *Lorentz and permutation invariants of particles I*, arXiv:2003.05487, doi:10.1088/1751-8121/abe58c.

[2] C. G. Lester, W. Haddadin and B. Gripaios, *Lorentz and permutation invariants of particles III: constraining non-standard sources of parity violation*, Int. J. Mod. Phys. A **37** (2022) 2250093, arXiv:2008.05206.

[3] V. Delle Rose, A. Kozachinskiy, C. Rojas, M. Petrache and P. Barceló, *Three iterations of $(d-1)$-WL test distinguish non isometric clouds of d-dimensional points*, arXiv:2303.12853.

[4] D. Widdowson and V. Kurlin, *Recognizing rigid patterns of unlabeled point clouds by complete and continuous isometry invariants with no false negatives and no false positives*, CVPR 2023.

[5] V. Kurlin, *Simplexwise distance distributions for finite spaces with metrics and measures*, arXiv:2303.14161.